How Hard is Taking Scope?

Ned Sanger

NYU Semantics Group

October 23, 2026

Plan

1. Scope-taking refresher
2. A little bit of complexity theory
3. Finding LFs is NP-complete
4. Coming back down to earth

Why scope-taking operations?

Why scope-taking operations?

Alice sheared every sheep

Why scope-taking operations?

Alice sheared every sheep

Syntactic composition: [*Alice*] + [*sheared every sheep*]

Why scope-taking operations?

Alice sheared every sheep

Syntactic composition: [*Alice*] + [*sheared every sheep*]

Semantic composition: [*every sheep*] + [*Alice sheared* __]

Why scope-taking operations?

Alice sheared every sheep

Syntactic composition: [*Alice*] + [*sheared every sheep*]

Semantic composition: [*every sheep*] + [*Alice sheared* __]

Two problems:

Why scope-taking operations?

Alice sheared every sheep

Syntactic composition: [*Alice*] + [*sheared every sheep*]

Semantic composition: [*every sheep*] + [*Alice sheared* __]

Two problems:

⇒ wrong pieces for assembling the meaning?

Why scope-taking operations?

Alice sheared every sheep

Syntactic composition: [*Alice*] + [*sheared every sheep*]

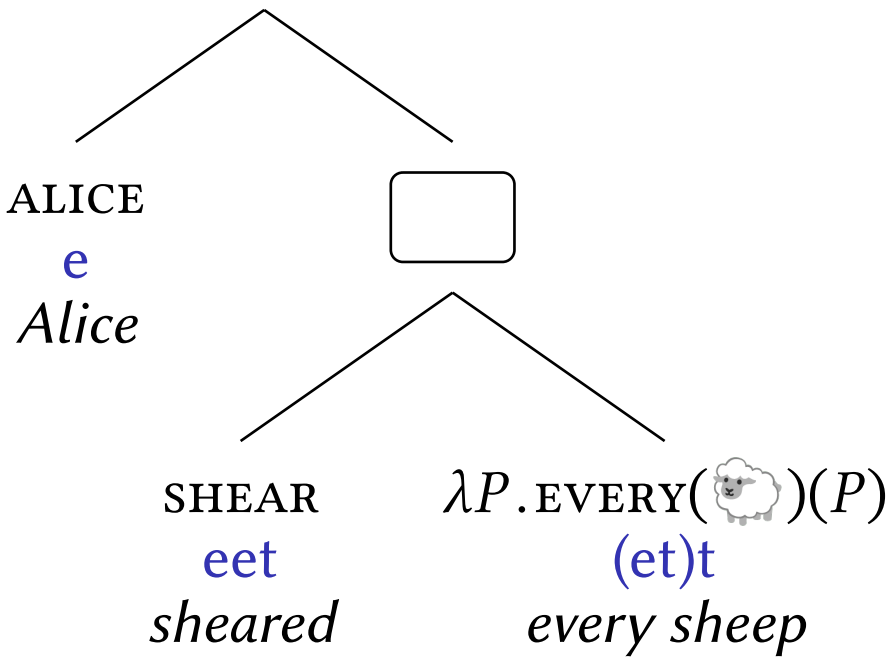
Semantic composition: [*every sheep*] + [*Alice sheared* __]

Two problems:

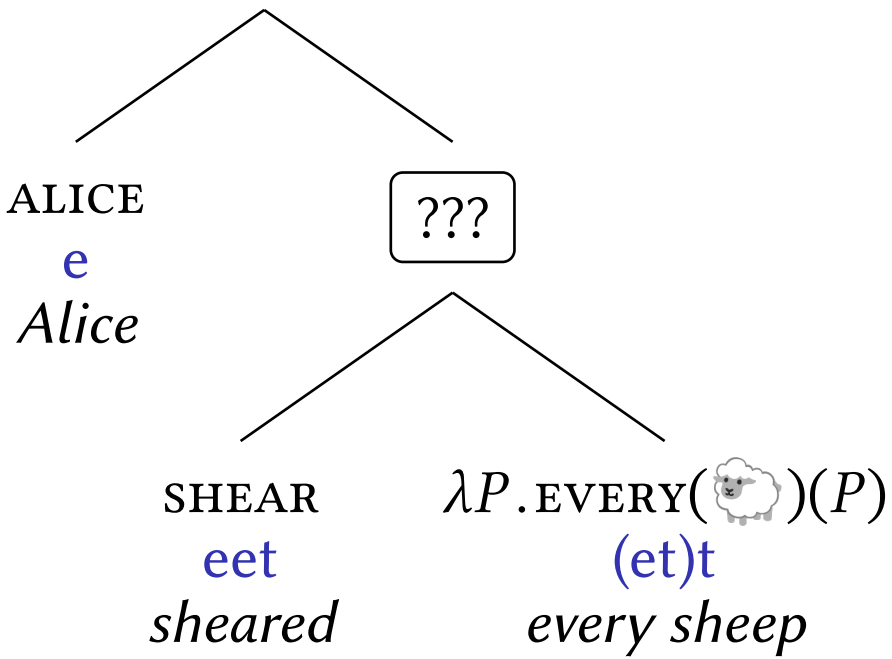
⇒ wrong pieces for assembling the meaning?

⇒ *every sheep* is higher-order: how to combine with *sheared*?

Why scope-taking operations?



Why scope-taking operations?



Why scope-taking operations?

Why scope-taking operations?

Lots of solutions:

Why scope-taking operations?

Lots of solutions:

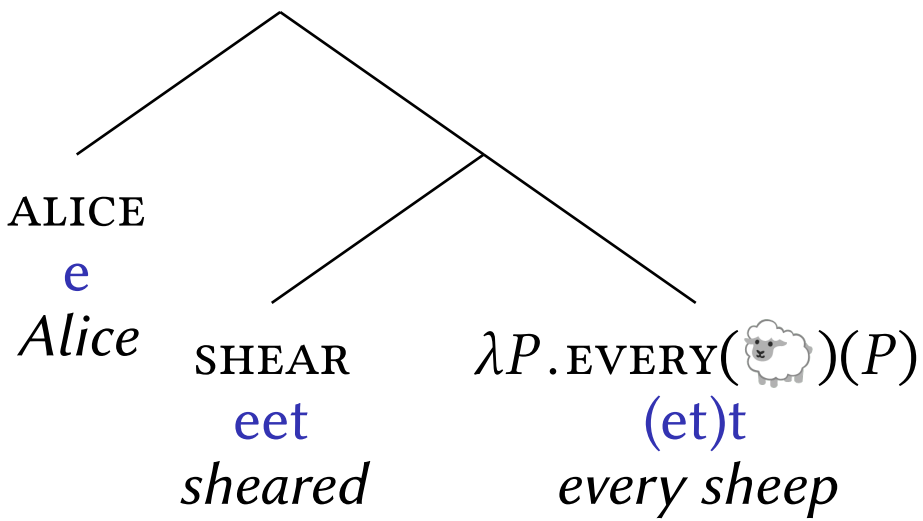
- Quantifier Raising (May 1977, Heim & Kratzer 1998)
- Quantifying In (Montague 1973)
- Cooper Storage (Cooper 1983)
- Continuations (Barker & Shan 2014, Kiselyov & Shan 2014)
- Flexible Montague Grammar (Hendriks 1993)
- …

Why scope-taking operations?

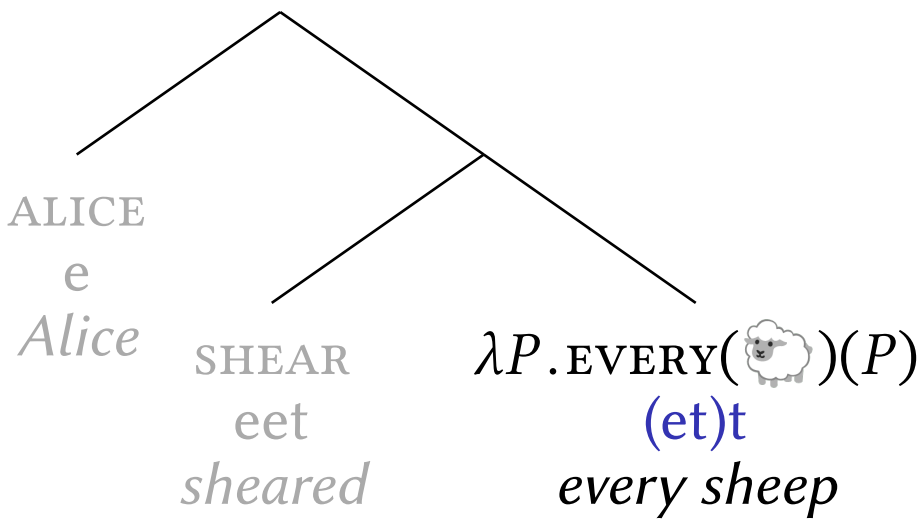
Lots of solutions:

- **Quantifier Raising** (May 1977, Heim & Kratzer 1998)
- Quantifying In (Montague 1973)
- Cooper Storage (Cooper 1983)
- Continuations (Barker & Shan 2014, Kiselyov & Shan 2014)
- Flexible Montague Grammar (Hendriks 1993)
- ...

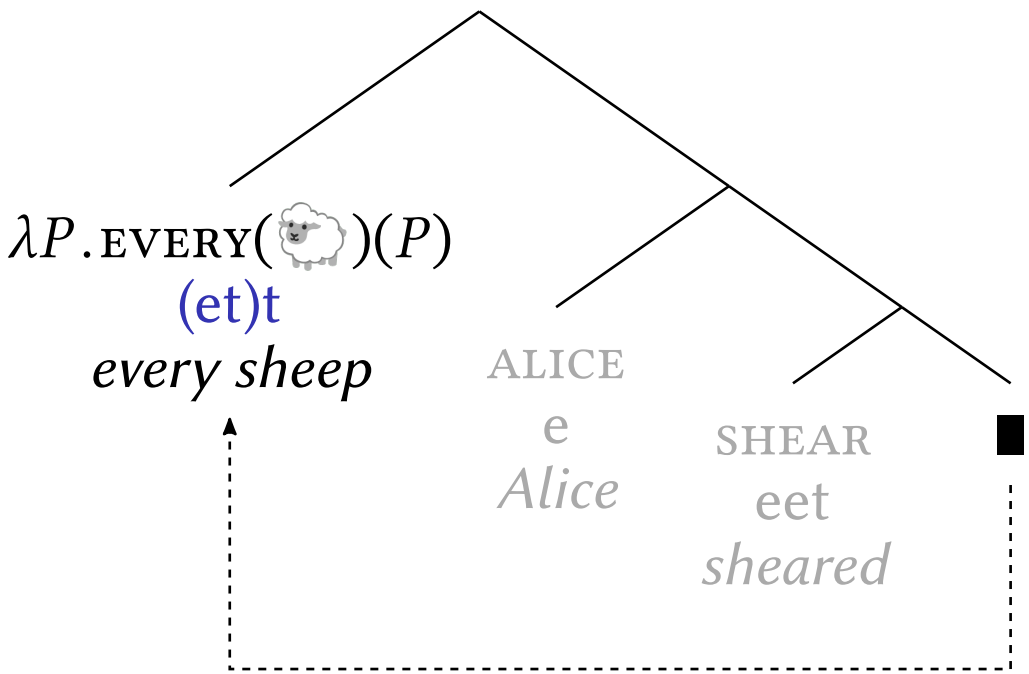
Quantifier Raising



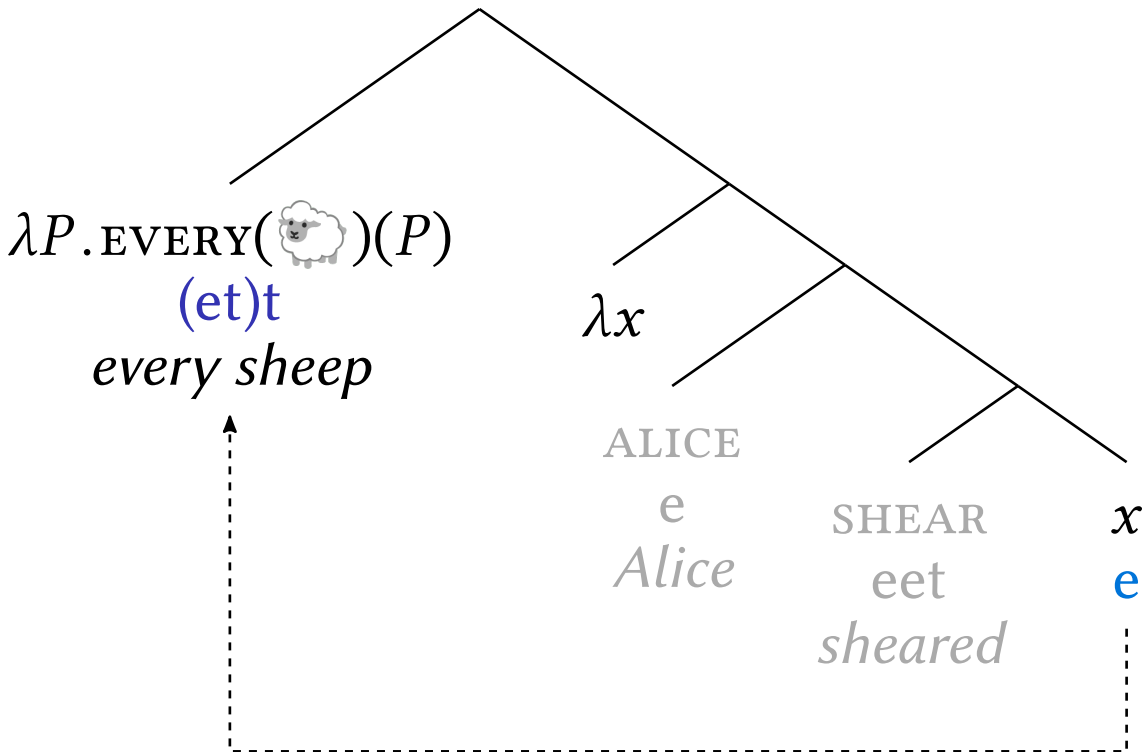
Quantifier Raising



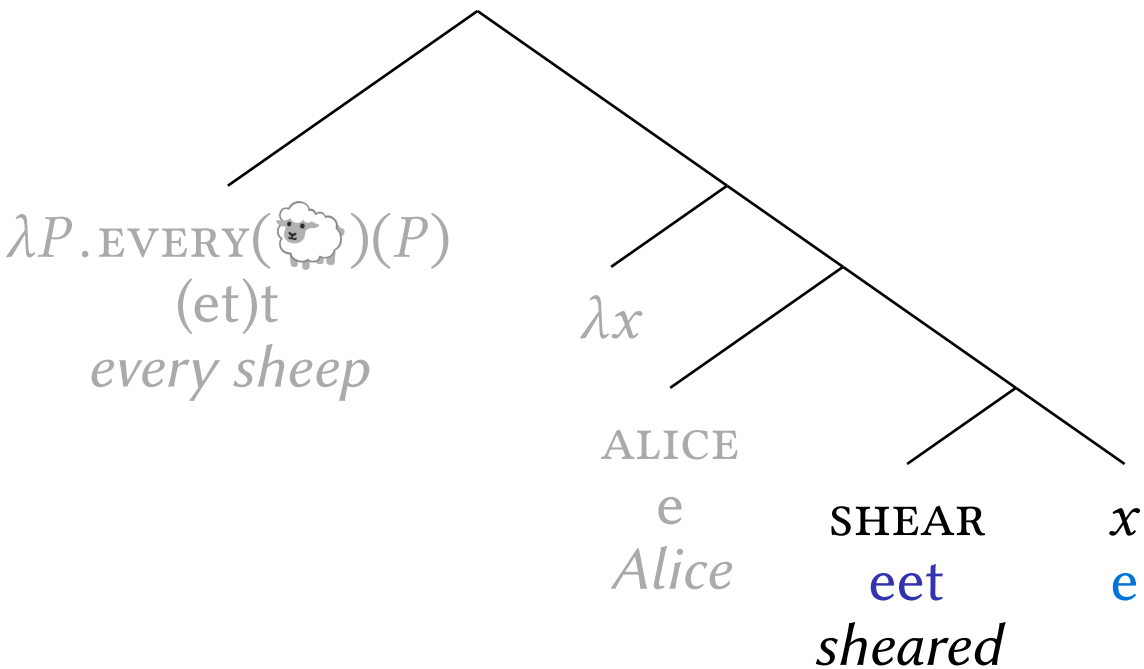
Quantifier Raising



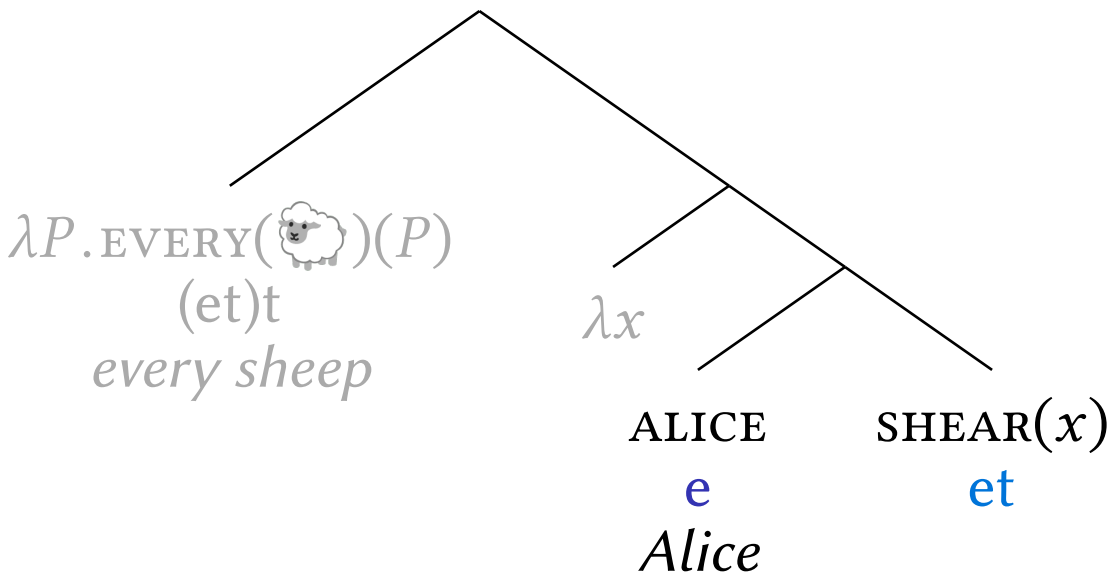
Quantifier Raising



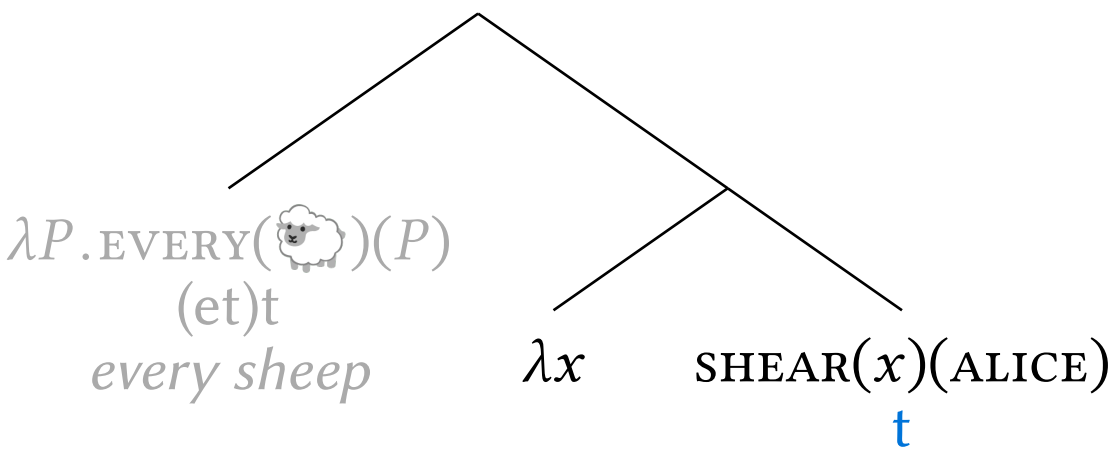
Quantifier Raising



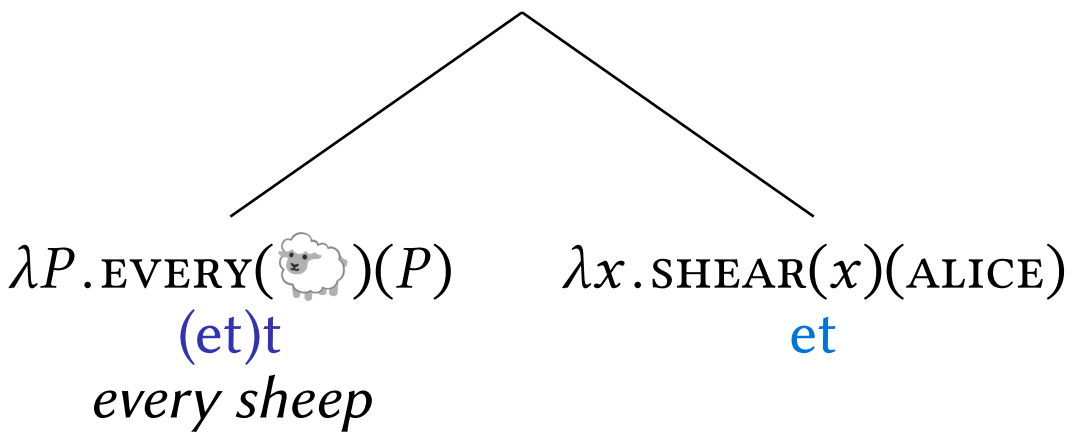
Quantifier Raising



Quantifier Raising



Quantifier Raising



Quantifier Raising

$$\textsc{every}(\text{🐑})(\lambda x.\,\textsc{shear}(x)(\textsc{alice}))$$

t

Quantifier Raising



Quantifier Raising

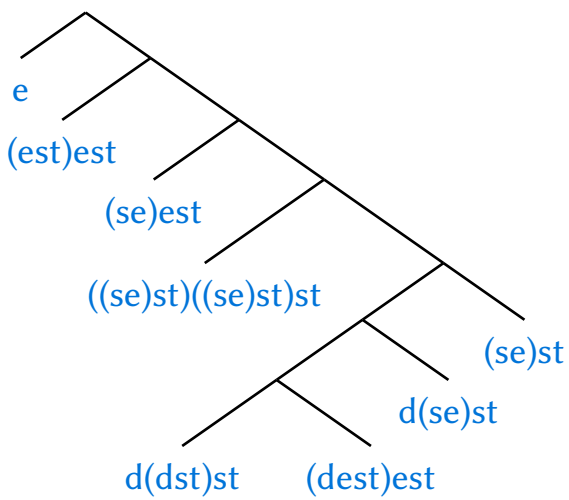
Is it always that easy?

Quantifier Raising

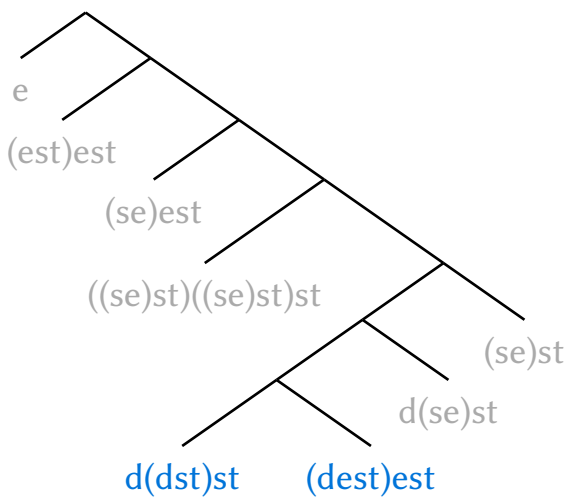
Ann has to climb the least high mountain

(cf. Hallman 2023)

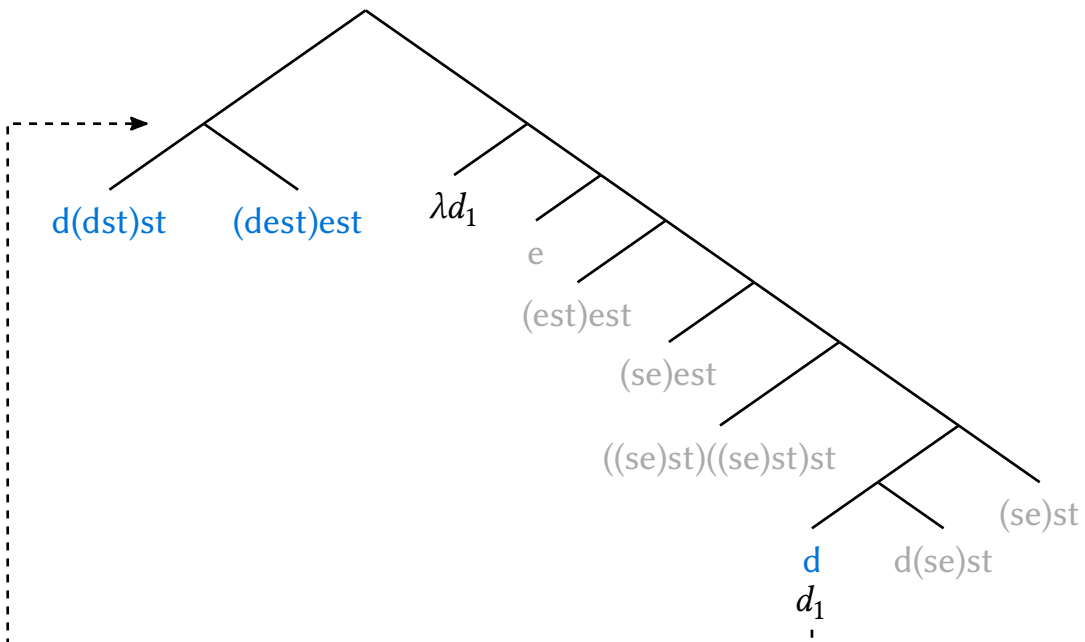
Quantifier Raising



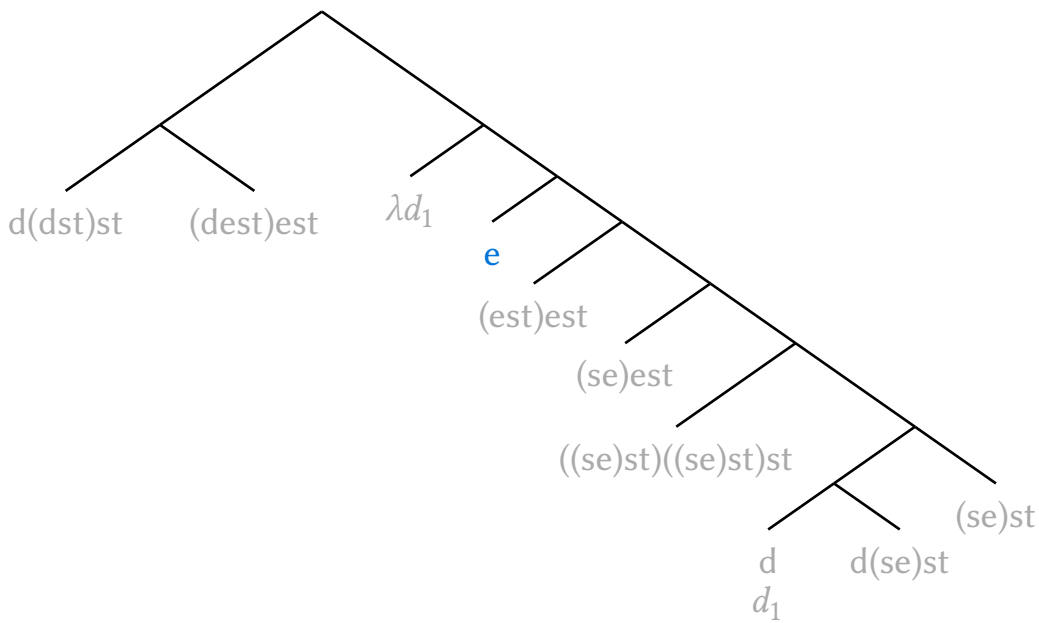
Quantifier Raising



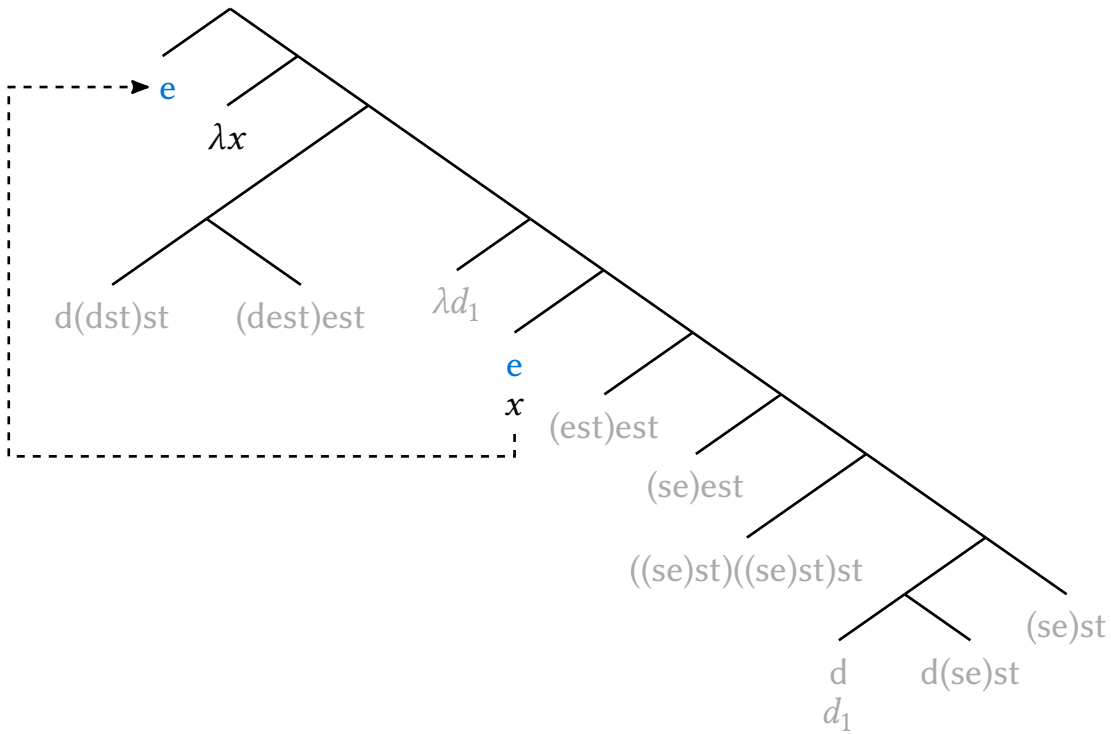
Quantifier Raising



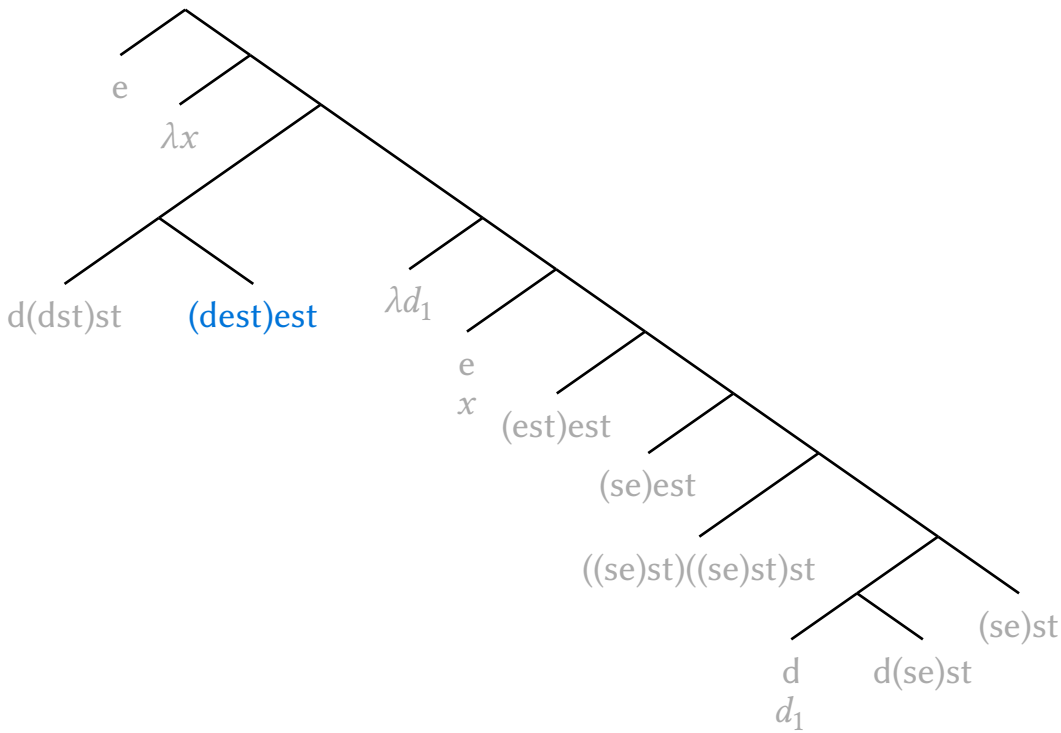
Quantifier Raising



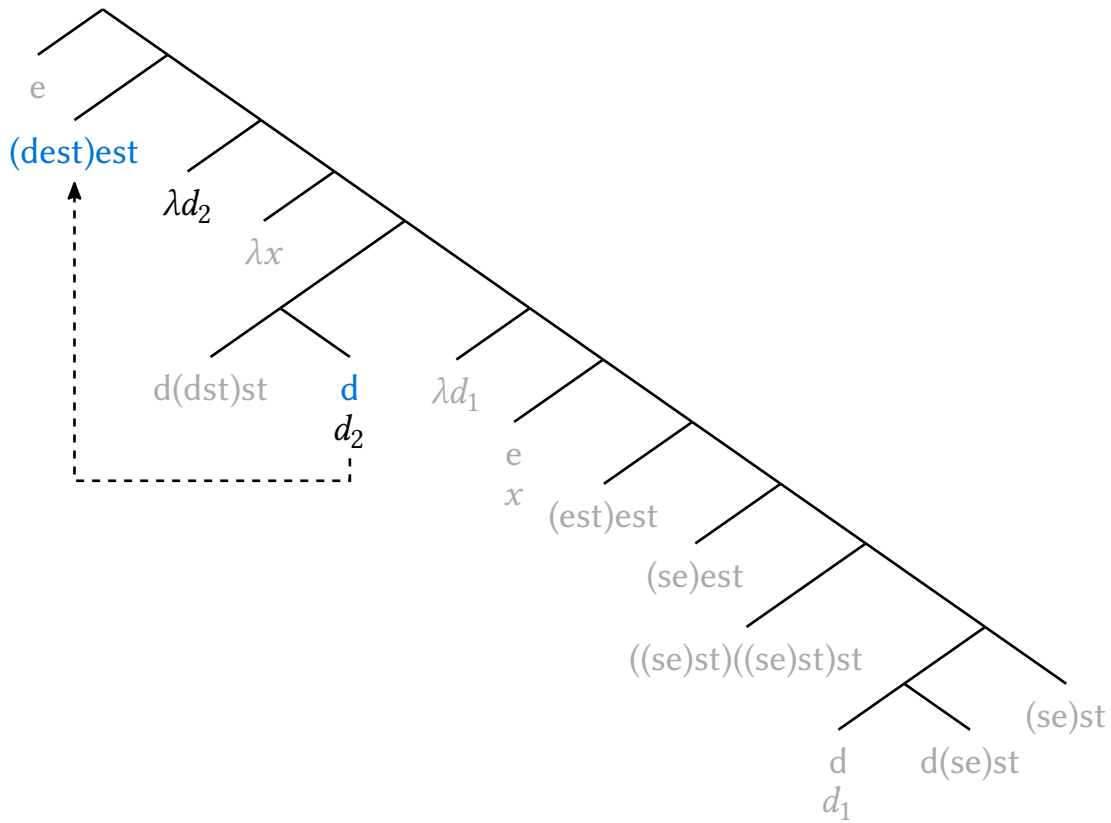
Quantifier Raising



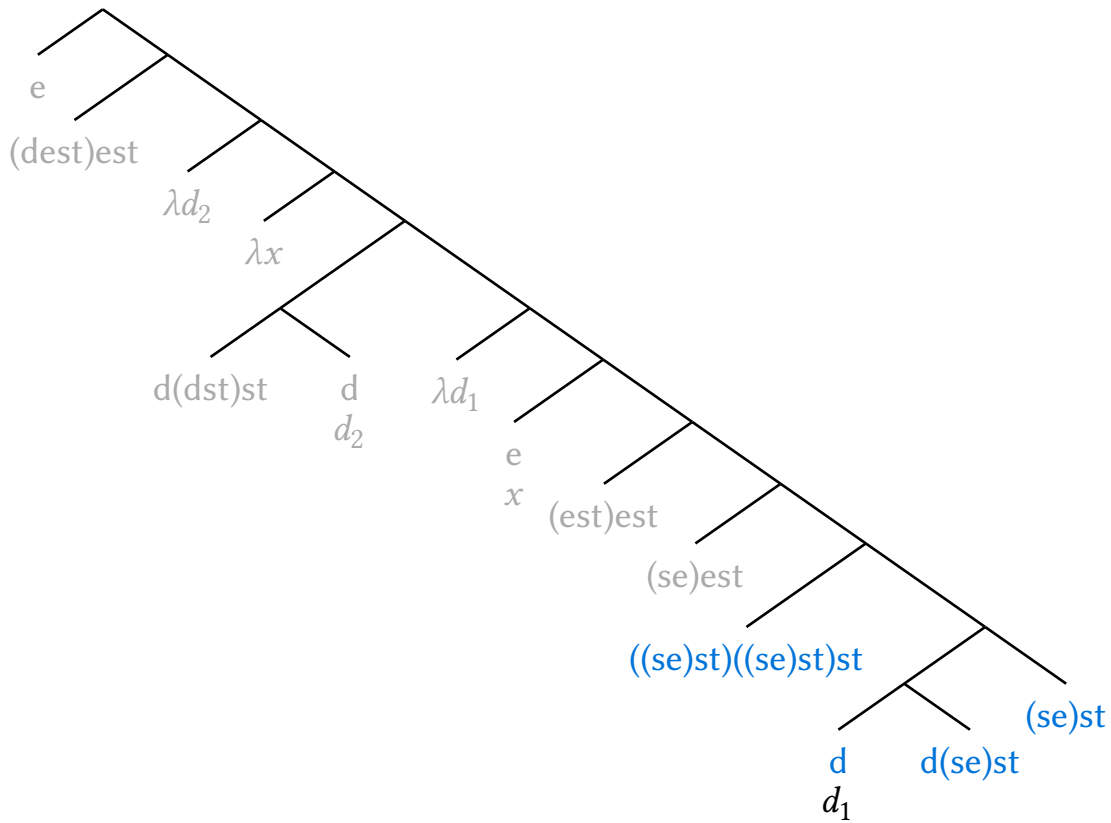
Quantifier Raising



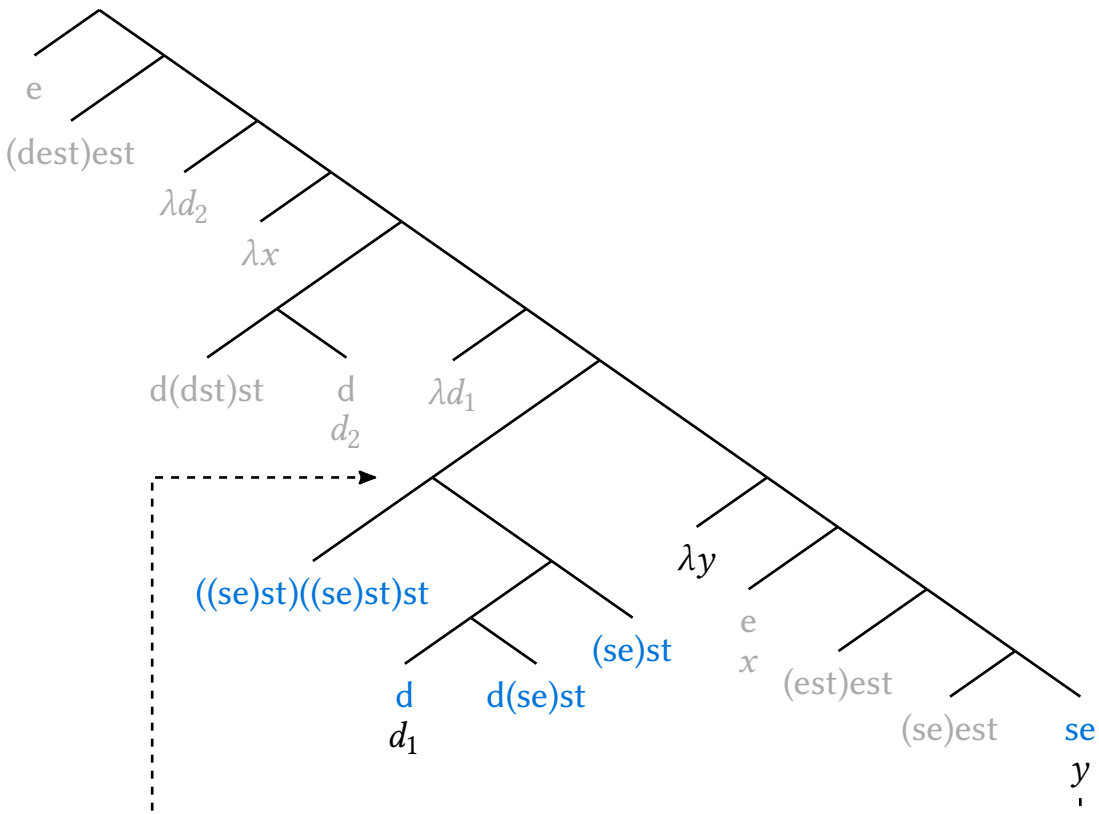
Quantifier Raising



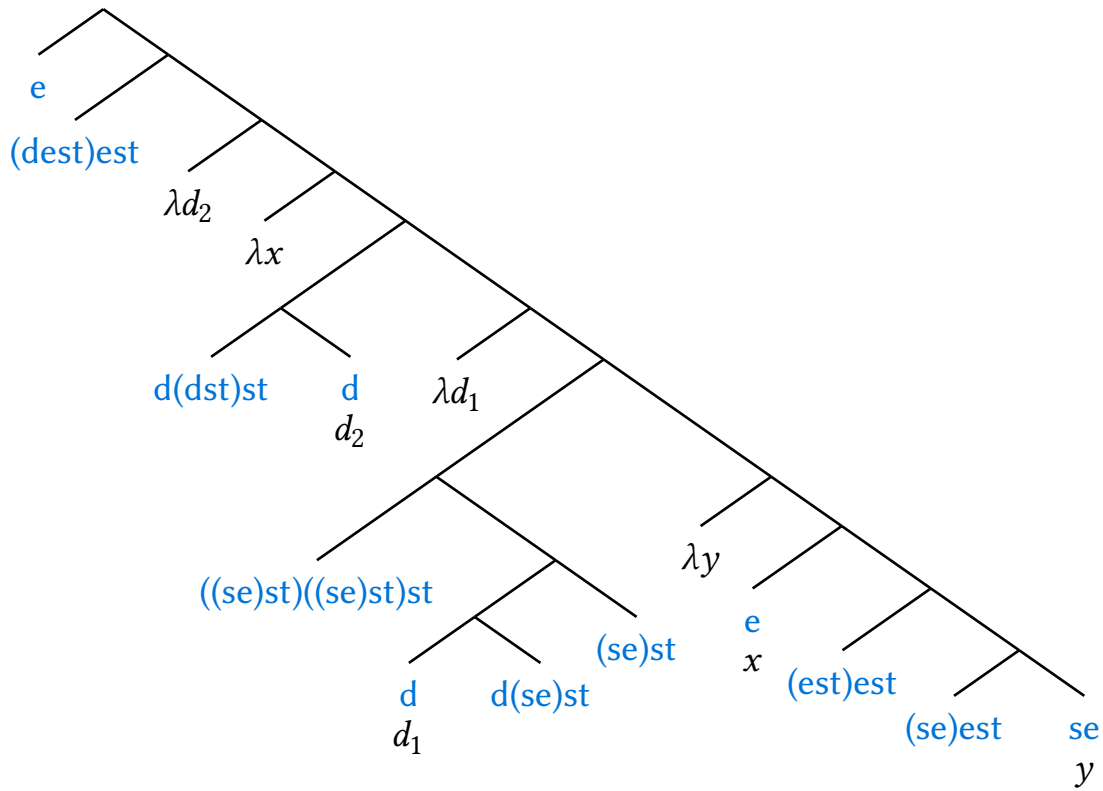
Quantifier Raising



Quantifier Raising



Quantifier Raising



Quantifier Raising

That was one LF. There are more.

Quantifier Raising

That was one LF. There are more.

Taking scope is complicated.

Quantifier Raising

Some reasons to study QR:

Quantifier Raising

Some reasons to study QR:

Practical: It would be useful to have fast algorithms that automate the search for LFs

Quantifier Raising

Some reasons to study QR:

Practical: It would be useful to have fast algorithms that automate the search for LFs

Theoretical: We should understand what computational mechanisms are necessary to use grammars with scope-taking

Quantifier Raising

Some reasons to study QR:

Practical: It would be useful to have fast algorithms that automate the search for LFs

Theoretical: We should understand what computational mechanisms are necessary to use grammars with scope-taking

⇒ If the space of LFs is efficiently explorable, that's an important fact about grammar

Quantifier Raising

Some reasons to study QR:

Practical: It would be useful to have fast algorithms that automate the search for LFs

Theoretical: We should understand what computational mechanisms are necessary to use grammars with scope-taking

⇒ If the space of LFs is efficiently explorable, that's an important fact about grammar

⇒ If it isn't, then what properties of the sentences we encounter make finding LFs (mostly) easy?

The Question

How hard is the following task?

The Question

How hard is the following task?

> Given a tree that is *not* well-typed, decide whether repeated uses of QR can turn it into one that *is*.

The Question

How hard is the following task?

> Given a tree that is *not* well-typed, decide whether repeated uses of QR can turn it into one that *is*.

Claim 1:

The Question

How hard is the following task?

> Given a tree that is *not* well-typed, decide whether repeated uses of QR can turn it into one that *is*.

Claim 1: This problem is intractable (NP-complete).

The Question

How hard is the following task?

> Given a tree that is *not* well-typed, decide whether repeated uses of QR can turn it into one that *is*.

Claim 1: This problem is intractable (NP-complete).

Claim 2:

The Question

How hard is the following task?

> Given a tree that is *not* well-typed, decide whether repeated uses of QR can turn it into one that *is*.

Claim 1: This problem is intractable (NP-complete).

Claim 2: This result is robust. It doesn't depend on:

The Question

How hard is the following task?

> Given a tree that is *not* well-typed, decide whether repeated uses of QR can turn it into one that *is*.

Claim 1: This problem is intractable (NP-complete).

Claim 2: This result is robust. It doesn't depend on: the number of atomic types, higher-order traces, parasitic scope, cyclic QR, ...

INTERLUDE

INTERLUDE

to give the quickest overview of complexity theory

Complexity theory characterizes what *computational resources* a computer needs to answer a yes/no question. The complexity recipe:

Complexity theory characterizes what *computational resources* a computer needs to answer a yes/no question. The complexity recipe:

⇒ Pick a model of computation (e.g. multi-tape Turing machines)

Complexity theory characterizes what *computational resources* a computer needs to answer a yes/no question. The complexity recipe:

⇒ Pick a model of computation (e.g. multi-tape Turing machines)

⇒ Pick a question, e.g.:

- Does this CFG generate this sentence?
- Is this candidate optimal for this constraint ranking and UR?
- Are these two Boolean formulas equivalent?

Complexity theory characterizes what *computational resources* a computer needs to answer a yes/no question. The complexity recipe:

⇒ Pick a model of computation (e.g. multi-tape Turing machines)

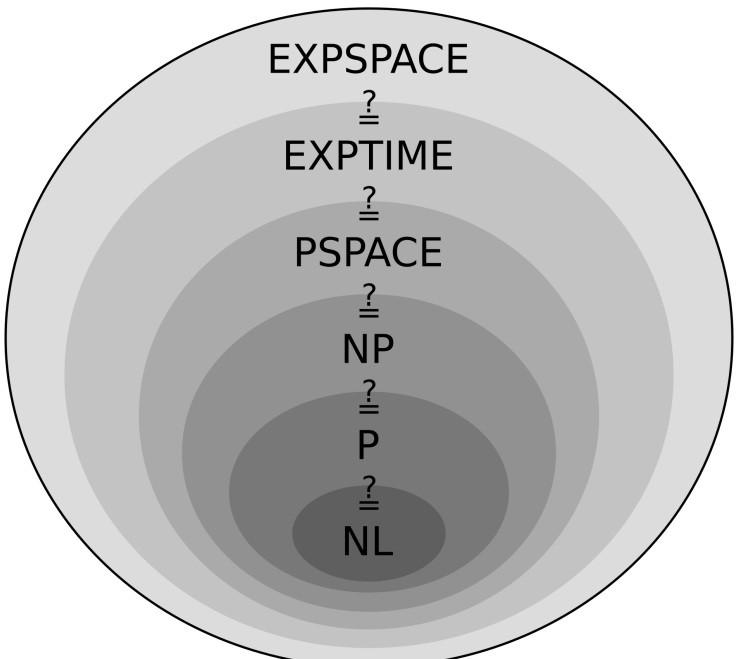
⇒ Pick a question, e.g.:

- Does this CFG generate this sentence?
- Is this candidate optimal for this constraint ranking and UR?
- Are these two Boolean formulas equivalent?

⇒ Determine how the time needed to compute an answer grows w.r.t. a parameter:

- length of the sentence, size of the grammar
- length of the candidate UR, number of constraints
- number of variables, number of connectives

Complexity Classes



All we care about today are P (efficiently solvable) and NP (efficiently verifiable)

NP-Complete Problems

The hardest problems in NP are called **NP-complete**.

NP-Complete Problems

The hardest problems in NP are called **NP-complete**.

- Thousands of well-known problems fall in this class
- They are computationally intractable
- I claim that finding a well-typed LF is one of them

NP-Complete Problems

The hardest problems in NP are called **NP-complete**.

- Thousands of well-known problems fall in this class
- They are computationally intractable
- I claim that finding a well-typed LF is one of them

How do we prove that a problem Q is NP-complete? Two steps:

- Show that Q is in NP: given a candidate solution, we can efficiently verify it
- Show that a problem R already known to be NP-complete can be easily rearranged to look like Q
  - If we could solve Q efficiently, then we could also solve R efficiently, so Q is at least as hard as R

END OF INTERLUDE

Notation

$\Gamma \rightarrow^{*}_{\mathrm{QR}} \Delta$ means Δ is derivable from Γ by applying QR zero or more times.

QR is allowed to raise *any* constituent to *any* c-commanding position as long as it doesn't:

- create an unbound trace,
- or raise a λ-abstraction.

LF Repair

The formal decision problem we're interested in:

LF Repair

Let:

- $\mathbf{A}$ be a set of atomic types,
- τ be a type over A,
- Γ be a tree whose leaves are types over A

Is there a tree Δ of type τ such that $\Gamma \to^*_{\mathrm{QR}} \Delta$?

Proof Obligations

To show LF Repair is NP-complete, I have to show two things:

- LF Repair is in NP
- LF Repair is NP-hard

Proof Obligations

To show LF Repair is NP-complete, I have to show two things:

- **LF Repair is in NP**
- LF Repair is NP-hard

LF Repair is in NP

CB did the hard work (2020), I reaped the benefits. The idea:

1. Define a logic called Quantifier Raising w/ Types (QRT)
2.

3.

LF Repair is in NP

CB did the hard work (2020), I reaped the benefits. The idea:

1. Define a logic called Quantifier Raising w/ Types (QRT)
2. Show that building a well-typed LF is the same as proving a certain statement in QRT
3.

LF Repair is in NP

CB did the hard work (2020), I reaped the benefits. The idea:

1. Define a logic called Quantifier Raising w/ Types (QRT)
2. Show that building a well-typed LF is the same as proving a certain statement in QRT
3. Use cut elimination to show QRT-provability is in NP

Quantifier Raising w/ Types

$$\frac{}{A \vdash A}\,\mathrm{Ax} \qquad \frac{\Gamma \vdash A \qquad \Delta[B] \vdash C}{\Delta[\Gamma \bullet AB] \vdash C}\,{\to}\mathrm{L} \qquad \frac{\Gamma[A] \vdash B}{\lambda x.\Gamma[x] \vdash AB}\,{\to}\mathrm{R}$$

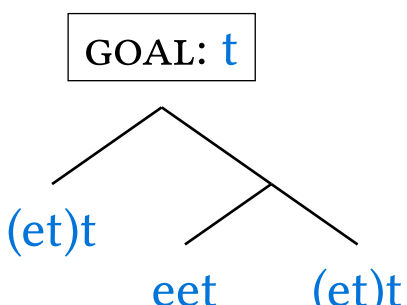
$$\frac{\Gamma[\Theta \bullet \lambda x.\Delta[x]] \vdash A}{\Gamma[\Delta[\Theta]] \vdash A}\,\lambda^{\uparrow} \qquad \frac{\Gamma[\Delta[\Theta]] \vdash A}{\Gamma[\Theta \bullet \lambda x.\Delta[x]] \vdash A}\,\lambda^{\downarrow}$$

$$\frac{\Gamma \vdash A \qquad \Delta[A] \vdash B}{\Delta[\Gamma] \vdash B}\,\textsc{Cut}$$

$\lambda^{\uparrow}$ and $\lambda^{\downarrow}$ are subject to the constraint that Θ is not a λ-abstraction.

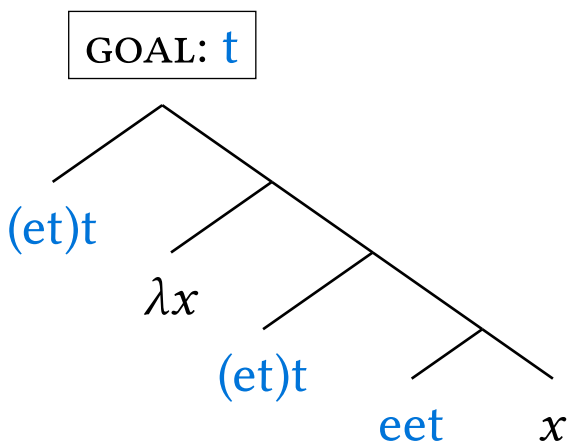
Example

$$(et)t \bullet (eet \bullet (et)t) \vdash t$$



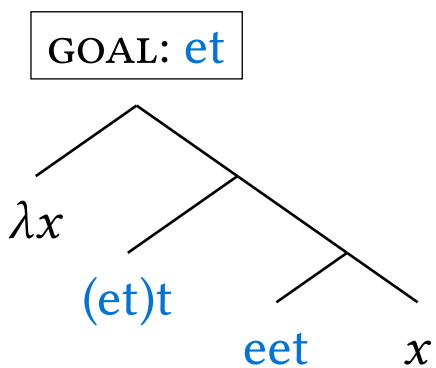
Example

$$\frac{(et)t \bullet \lambda x.((et)t \bullet (eet \bullet x)) \vdash t}{(et)t \bullet (eet \bullet (et)t) \vdash t}\ \lambda\uparrow$$



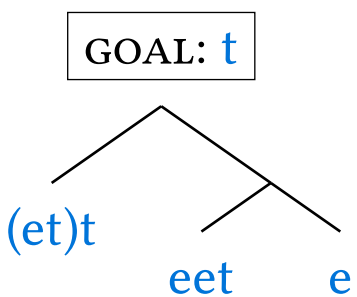
Example

$$\dfrac{\dfrac{\lambda x.((et)t \bullet (eet \bullet x)) \vdash et \qquad \dfrac{}{t \vdash t}\,\mathrm{Ax}}{(et)t \bullet \lambda x.((et)t \bullet (eet \bullet x)) \vdash t}\,{\to}\mathrm{L}}{(et)t \bullet (eet \bullet (et)t) \vdash t}\,\lambda{\uparrow}$$



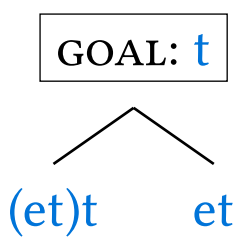
Example

$$\dfrac{\dfrac{\dfrac{(et)t \bullet (eet \bullet e) \vdash t}{\lambda x.((et)t \bullet (eet \bullet x)) \vdash et}\,{\to}\mathrm{R} \qquad \dfrac{}{t \vdash t}\,\mathrm{Ax}}{(et)t \bullet \lambda x.((et)t \bullet (eet \bullet x)) \vdash t}\,{\to}\mathrm{L}}{(et)t \bullet (eet \bullet (et)t) \vdash t}\,\lambda{\uparrow}$$



Example

$$\dfrac{\dfrac{\dfrac{\dfrac{}{e \vdash e}\,\mathrm{Ax} \qquad \dfrac{}{(et)t \bullet et \vdash t}\,\mathrm{Ax}}{(et)t \bullet (eet \bullet e) \vdash t}\,{\to}\mathrm{L}}{\lambda x.((et)t \bullet (eet \bullet x)) \vdash et}\,{\to}\mathrm{R} \qquad \dfrac{}{t \vdash t}\,\mathrm{Ax}}{(et)t \bullet \lambda x.((et)t \bullet (eet \bullet x)) \vdash t}\,{\to}\mathrm{L}}{(et)t \bullet (eet \bullet (et)t) \vdash t}\,\lambda{\uparrow}$$



Example

$$\dfrac{\dfrac{\dfrac{}{e \vdash e}\,\mathrm{Ax} \qquad \dfrac{\dfrac{}{et \vdash et}\,\mathrm{Ax} \qquad \dfrac{}{e \vdash e}\,\mathrm{Ax}}{(et)t \bullet et \vdash t}\,{\to}\mathrm{L}}{(et)t \bullet (eet \bullet e) \vdash t}\,{\to}\mathrm{L}}{\lambda x.((et)t \bullet (eet \bullet x)) \vdash et}\,{\to}\mathrm{R} \qquad \dfrac{}{t \vdash t}\,\mathrm{Ax}}{(et)t \bullet \lambda x.((et)t \bullet (eet \bullet x)) \vdash t}\,{\to}\mathrm{L}$$
$$\dfrac{(et)t \bullet \lambda x.((et)t \bullet (eet \bullet x)) \vdash t}{(et)t \bullet (eet \bullet (et)t) \vdash t}\,\lambda{\uparrow}$$

GOAL: t

t

LF Repair is in NP

Theorem 1: The rules Cut and $\lambda^{\downarrow}$ are eliminable

LF Repair is in NP

Theorem 1: The rules Cut and $\lambda^{\downarrow}$ are eliminable

Corollary 1: Every QRT theorem $\Gamma \vdash A$ has a proof polynomial in the size of Γ and A

LF Repair is in NP

Theorem 1: The rules Cut and $\lambda^{\downarrow}$ are eliminable

Corollary 1: Every QRT theorem $\Gamma \vdash A$ has a proof polynomial in the size of Γ and A

Theorem 2: $\Gamma \vdash A$ is a theorem in QRT iff $\Gamma \rightarrow^{*}_{\mathrm{QR}} \Delta$ for a Δ of type A

LF Repair is in NP

Theorem 1: The rules Cut and $\lambda^{\downarrow}$ are eliminable

Corollary 1: Every QRT theorem $\Gamma \vdash A$ has a proof polynomial in the size of Γ and A

Theorem 2: $\Gamma \vdash A$ is a theorem in QRT iff $\Gamma \rightarrow^{*}_{\mathrm{QR}} \Delta$ for a Δ of type A

Corollary 2: LF Repair is in NP

Proof Obligations

To show LF Repair is NP-complete, I have to show two things:

- LF Repair is in NP
- LF Repair is NP-hard

Proof Obligations

To show LF Repair is NP-complete, I have to show two things:

- LF Repair is in NP
- **LF Repair is NP-hard**

LF Repair is NP-hard

Strategy: Show that finding well-typed LFs is equivalent to Directed Hamiltonian Cycle, a problem from graph theory

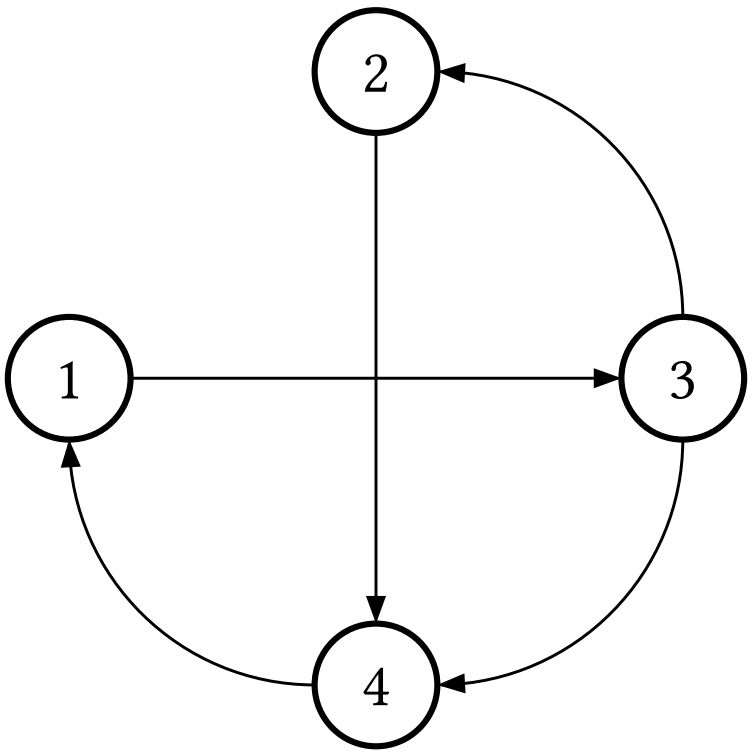
Hamiltonian Cycles

A *Hamiltonian cycle* is a cycle in a graph that visits every vertex exactly once. The following problem is NP-complete:

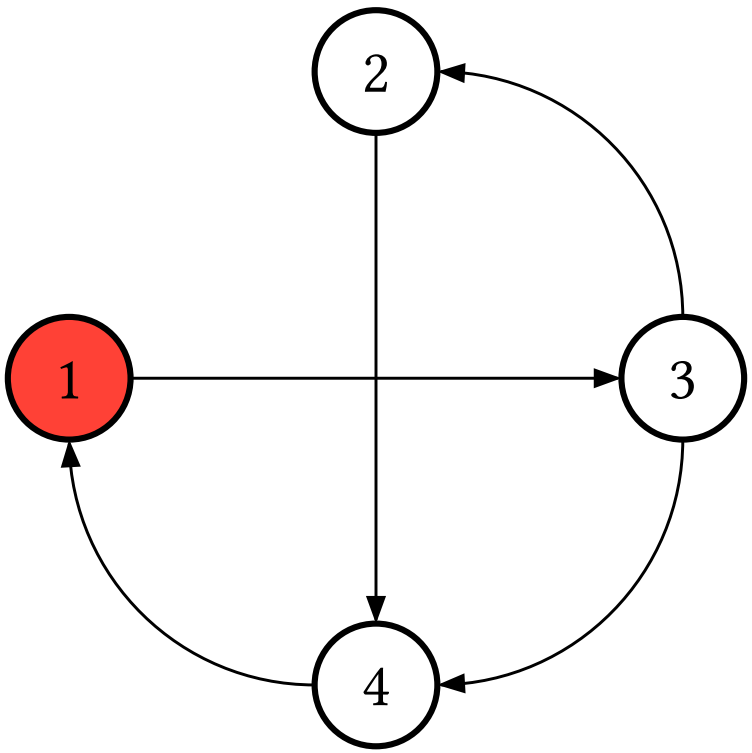
Directed Hamiltonian Cycle

Given a digraph, does it have a Hamiltonian cycle?

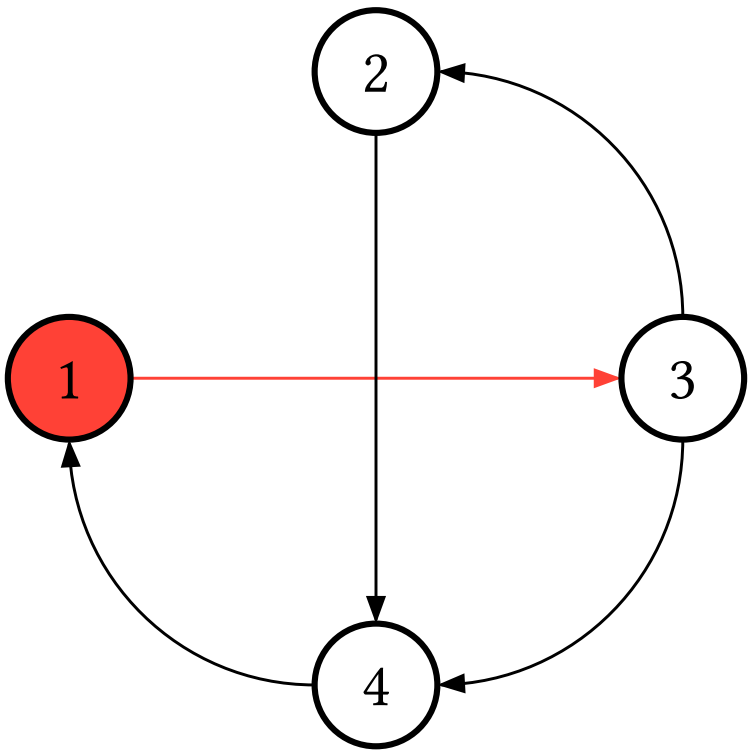
Hamiltonian Cycles



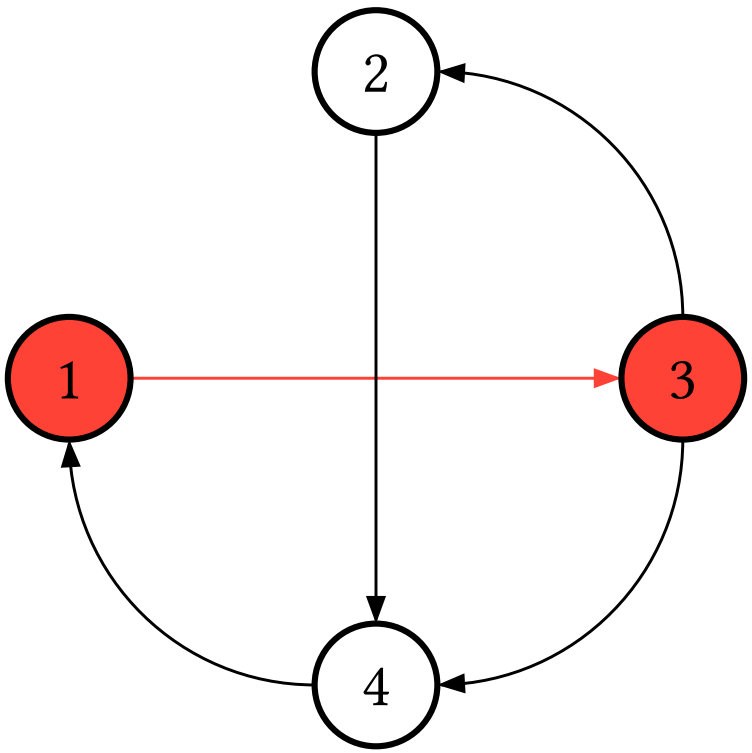
Hamiltonian Cycles



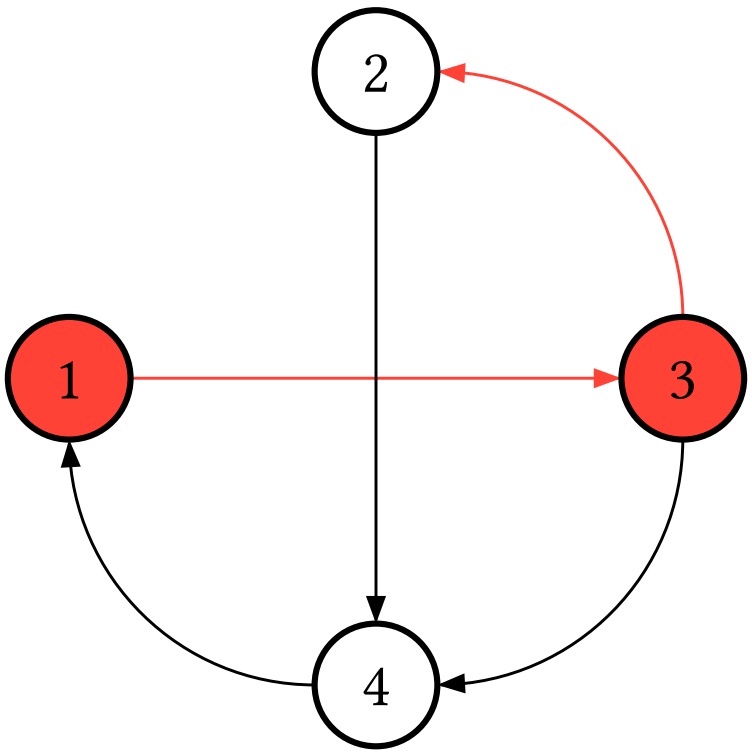
Hamiltonian Cycles



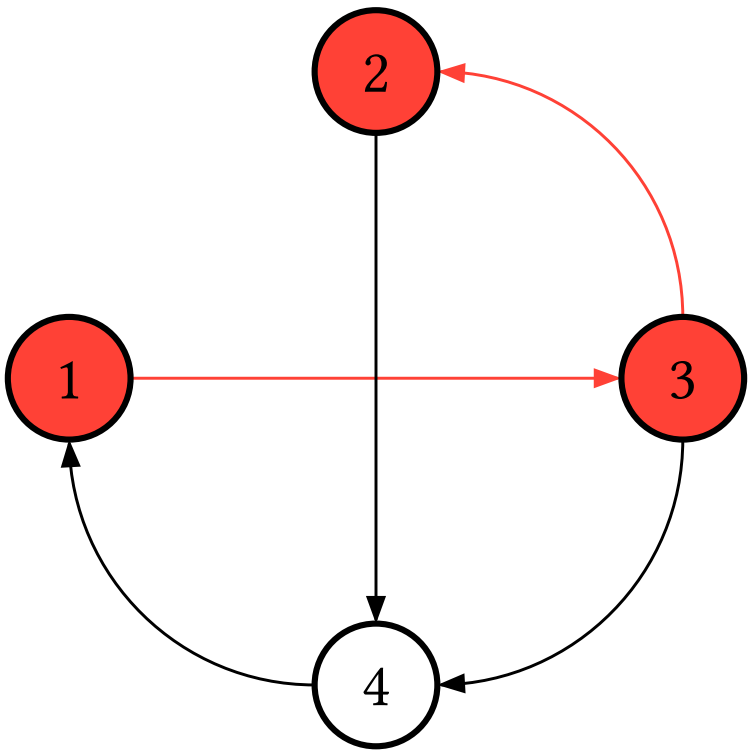
Hamiltonian Cycles



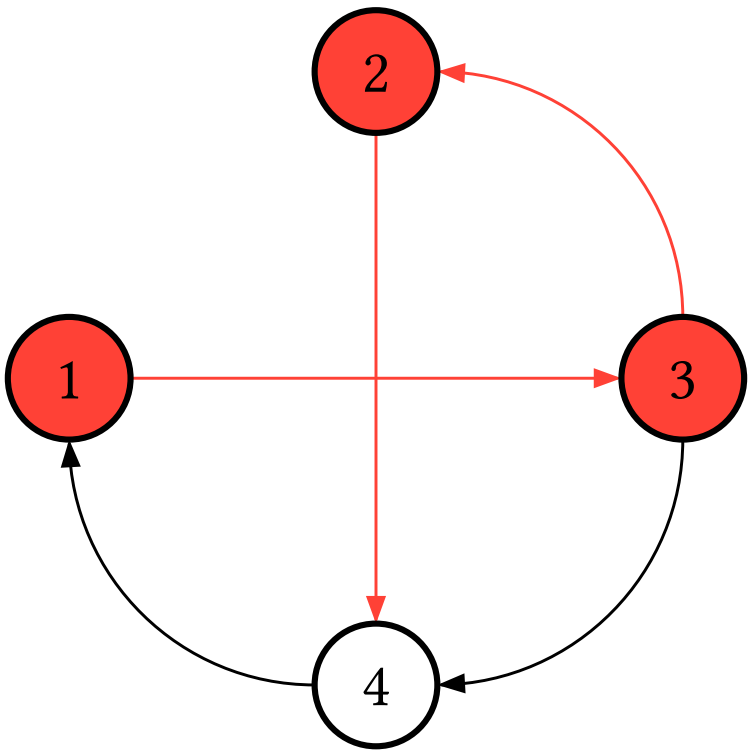
Hamiltonian Cycles



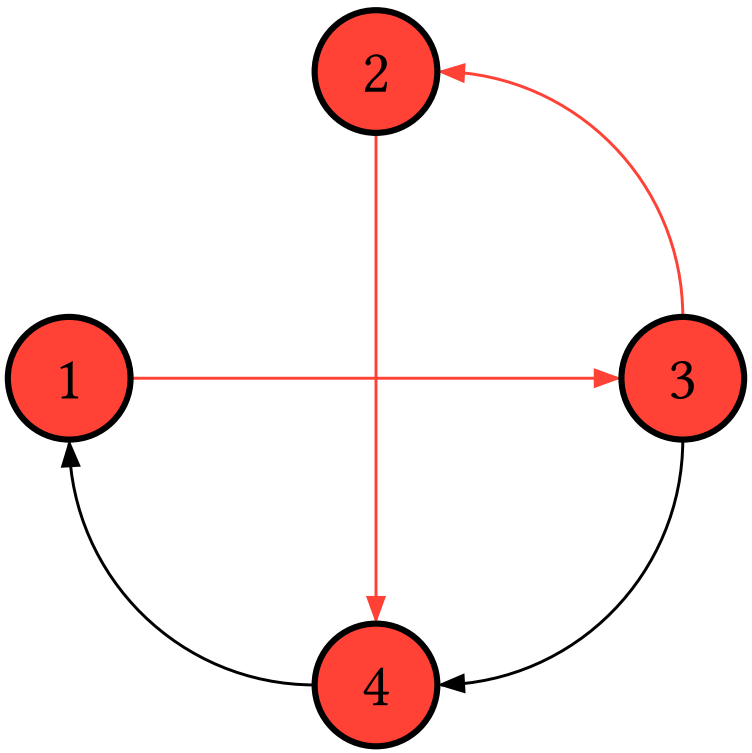
Hamiltonian Cycles



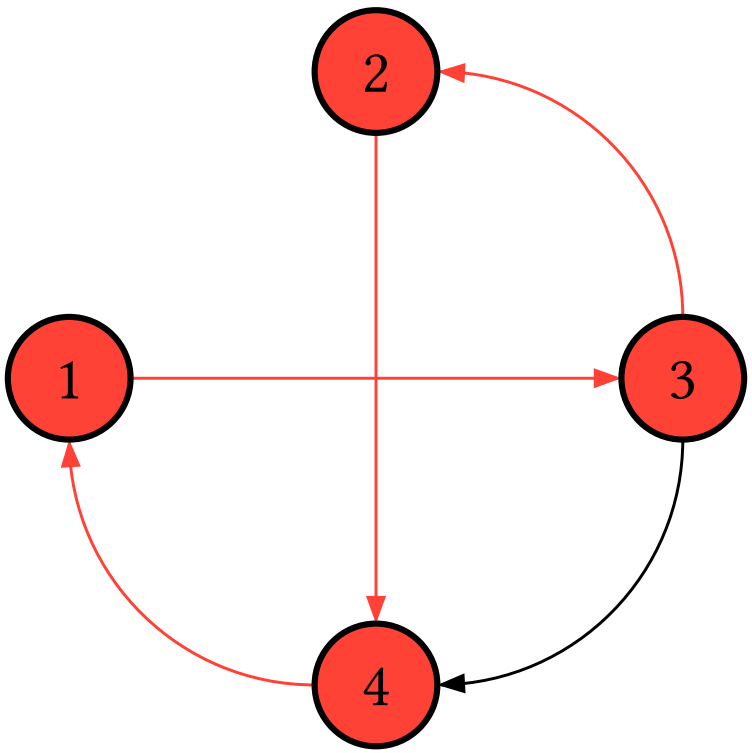
Hamiltonian Cycles



Hamiltonian Cycles



Hamiltonian Cycles



LF Repair is NP-hard

Claim: Given *any* digraph G, I can build an instance of LF Repair whose answer is *yes* iff G has a Hamiltonian Cycle

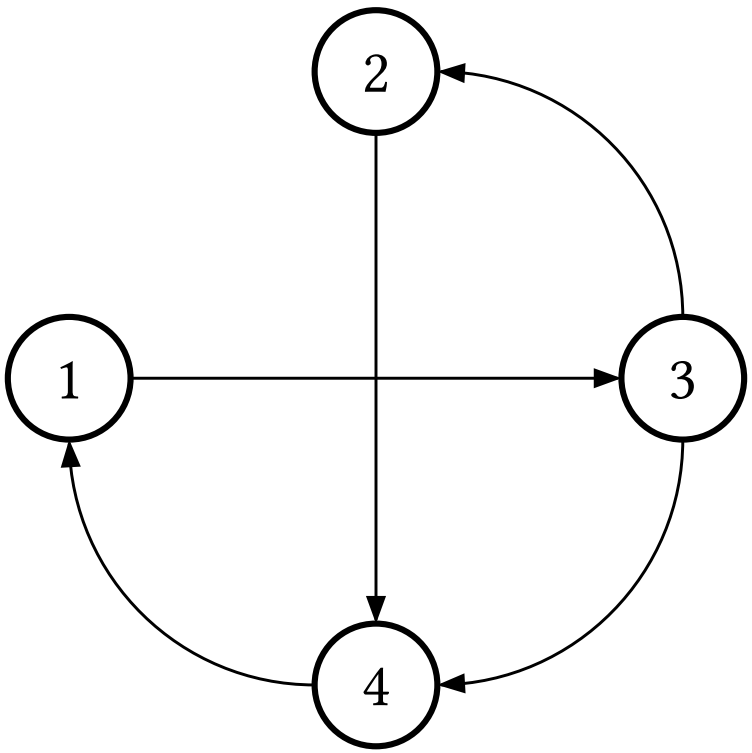
LF Repair is NP-hard

Claim: Given *any* digraph G, I can build an instance of LF Repair whose answer is *yes* iff G has a Hamiltonian Cycle

Idea:

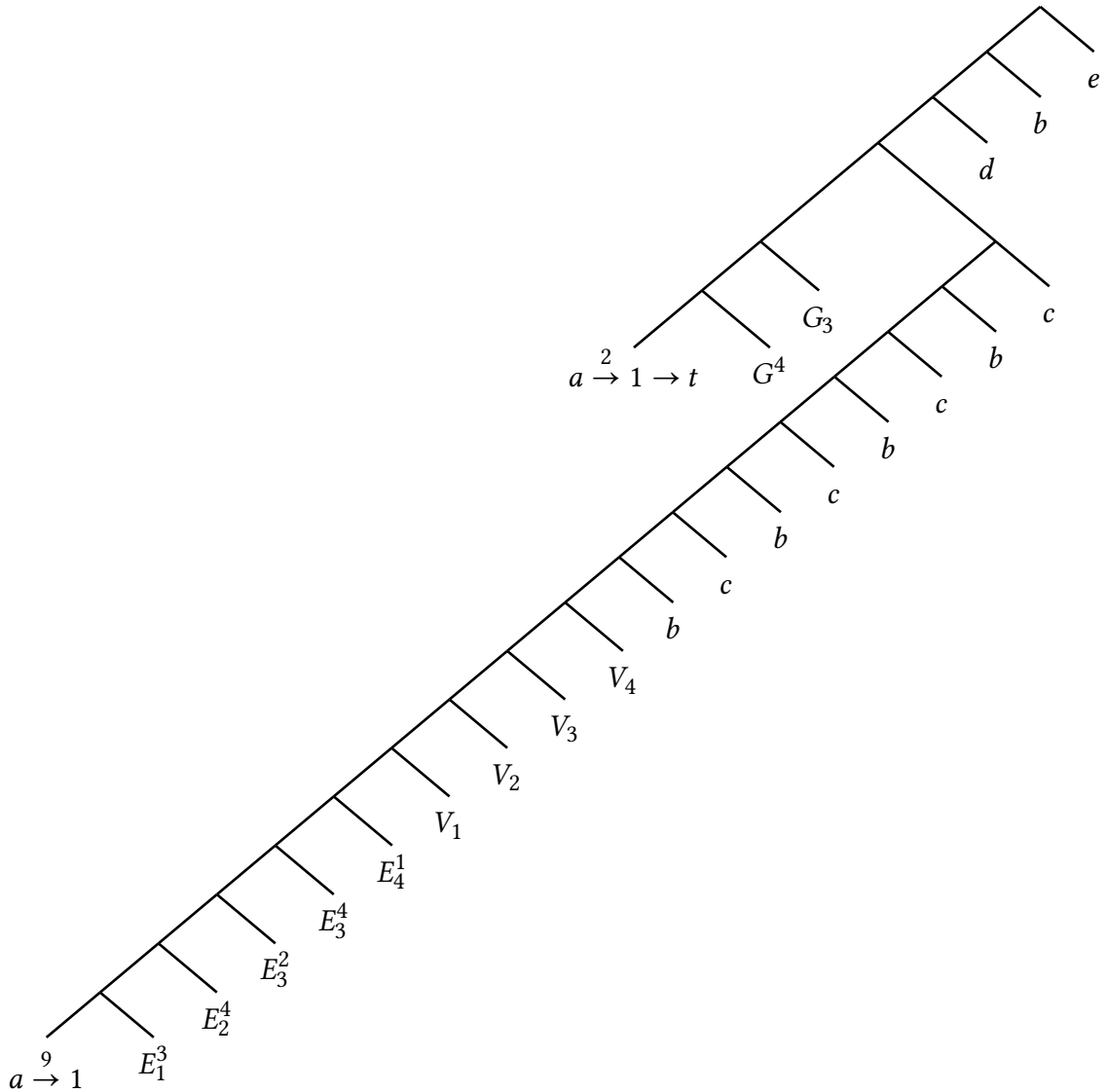
- have a type v for each vertex of the graph
- scope-takers of type $(__ \to u) \to v$ simulate moving along an edge from u to v
- chaining together a series of scope-takers simulates a path
- design the tree so that the only well-typed LF comes from chaining together scope-takers in a Hamiltonian cycle

LF Repair is NP-hard



LF Repair is NP-hard

drumroll



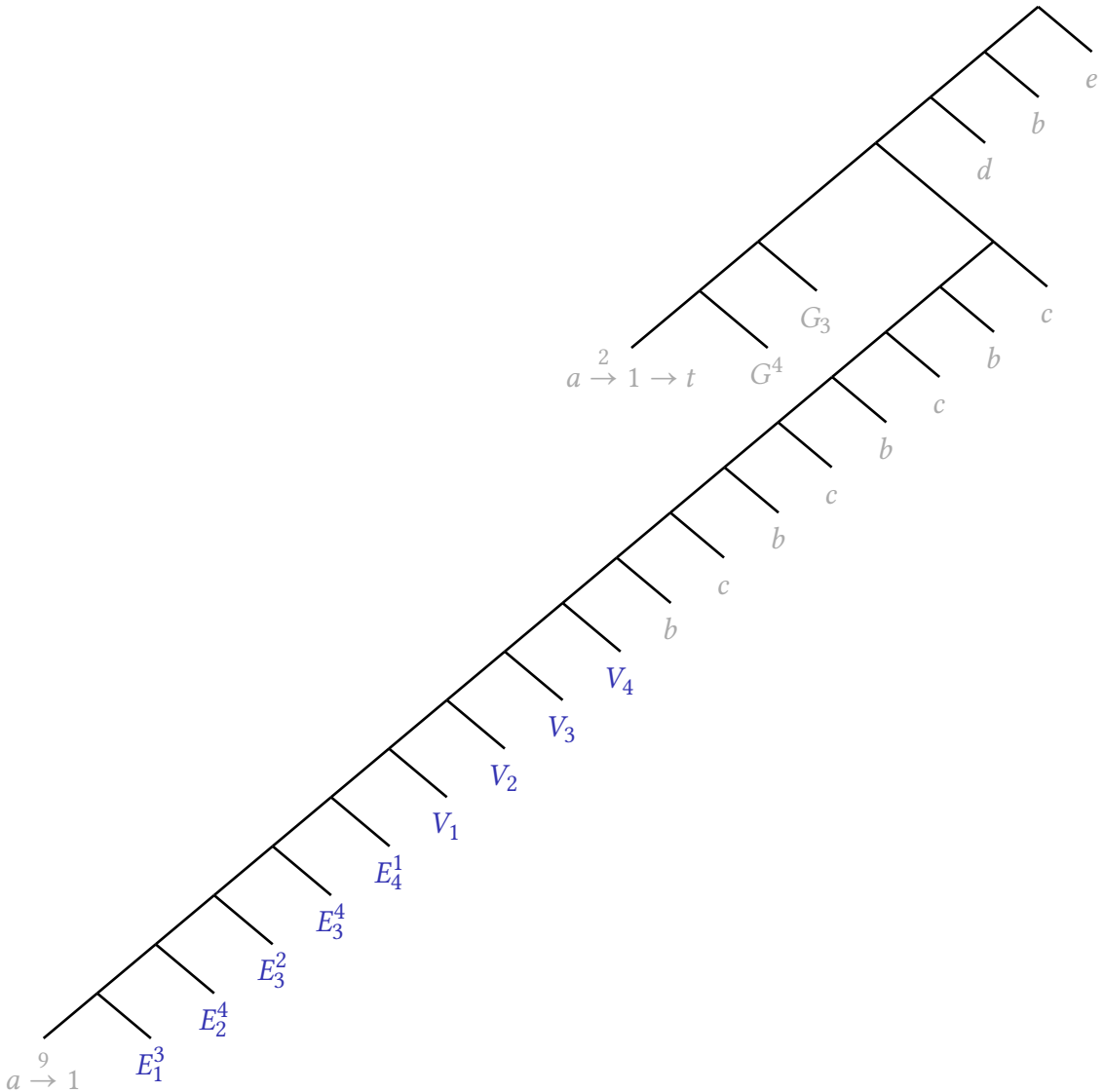
LEGEND

$E_u^v \coloneqq (\mathrm{a} \to u) \to \mathrm{b} \to v,$

$V_v \coloneqq (\mathrm{a} \to v) \to \mathrm{c} \to v,$

$G_v \coloneqq (\mathrm{a} \to \mathrm{t}) \to \mathrm{d} \to v,$

$G^v \coloneqq (\mathrm{a} \to v) \to \mathrm{e} \to \mathrm{t}.$



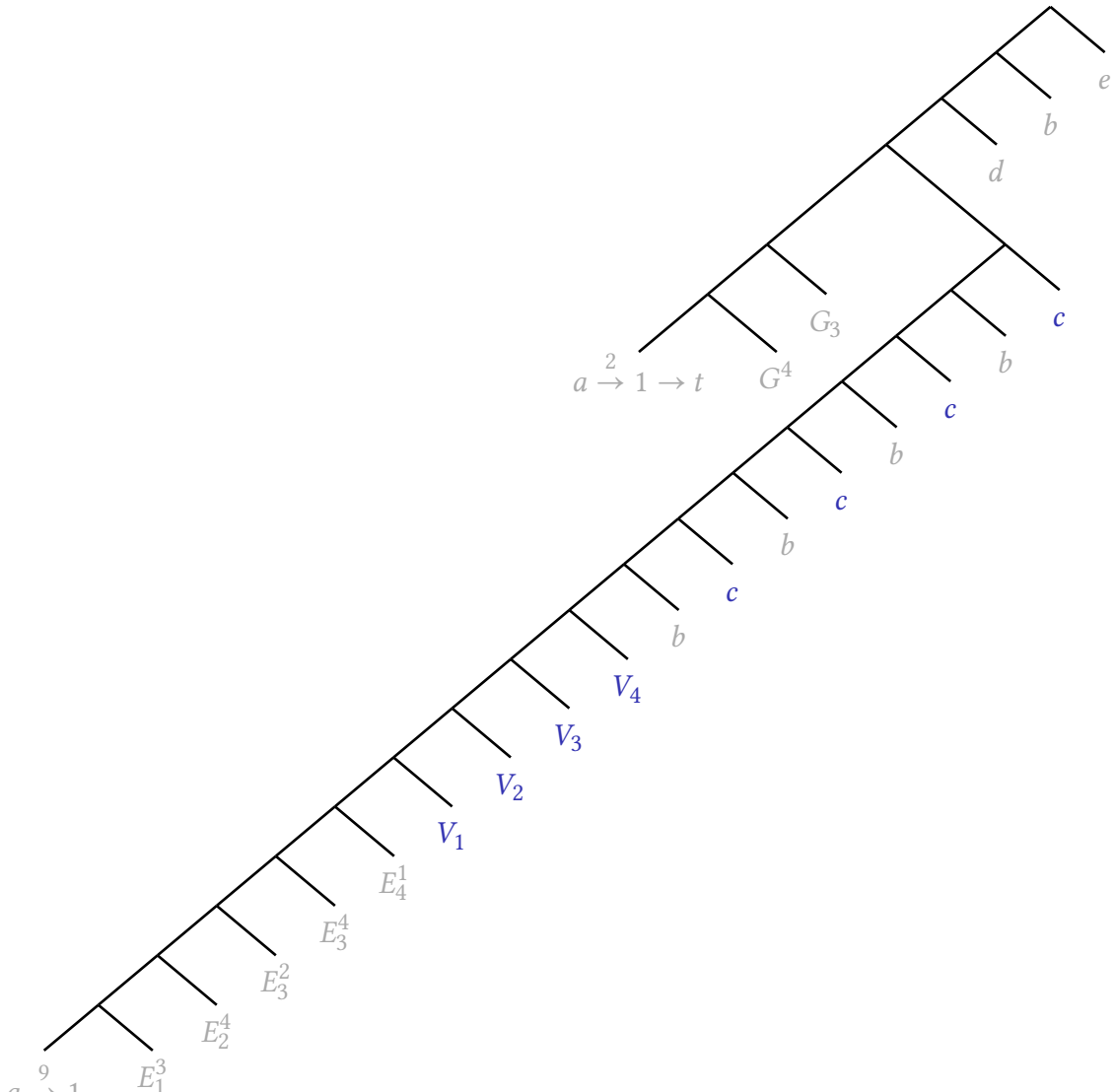
LEGEND

$E_u^v := (\mathrm{a} \to u) \to \mathrm{b} \to v,$

$V_v := (\mathrm{a} \to v) \to \mathrm{c} \to v,$

$G_v := (\mathrm{a} \to \mathrm{t}) \to \mathrm{d} \to v,$

$G^v := (\mathrm{a} \to v) \to \mathrm{e} \to \mathrm{t}.$



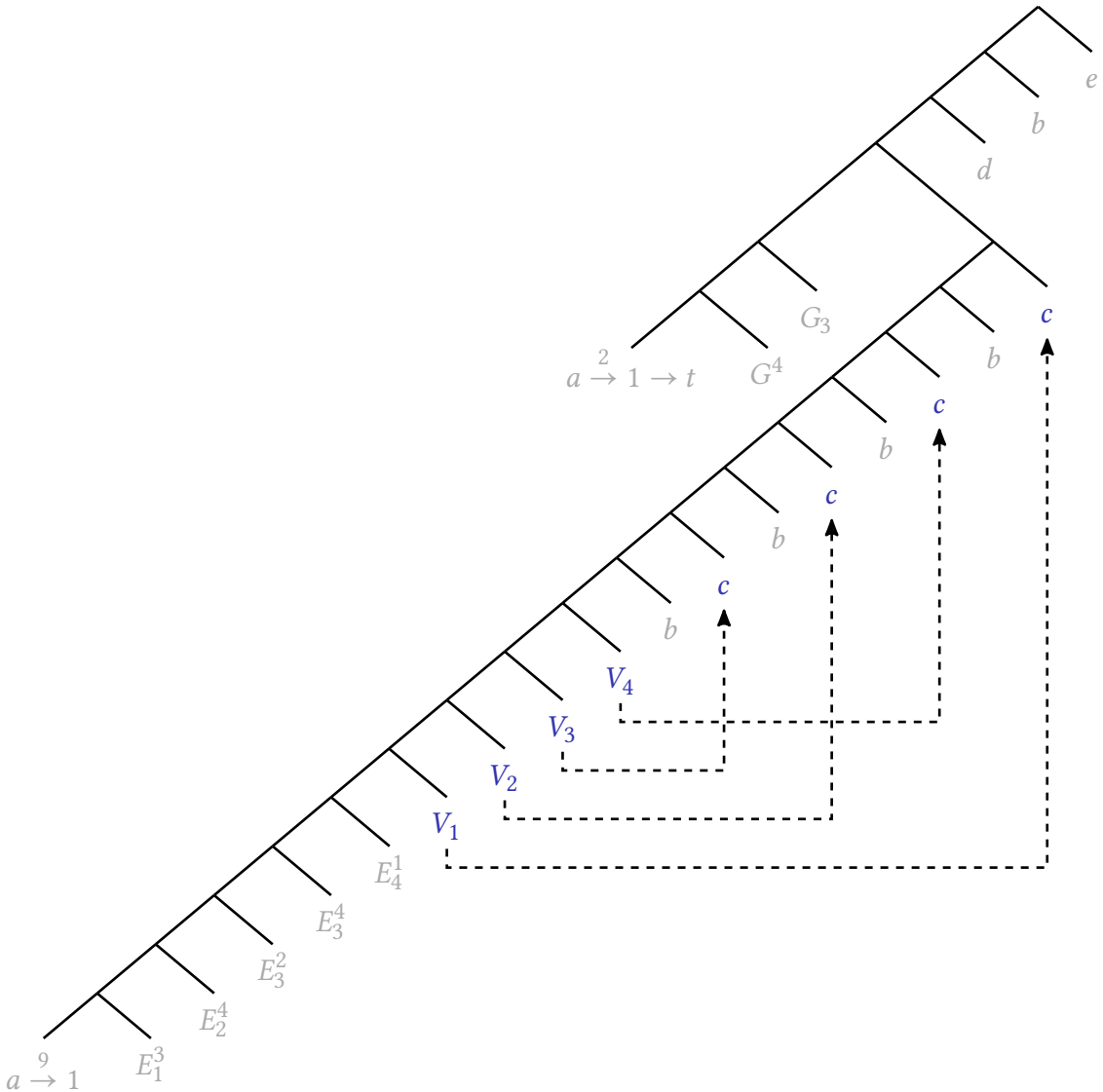
LEGEND

$E_u^v \coloneqq (\mathrm{a} \to u) \to \mathrm{b} \to v,$

$V_v \coloneqq (\mathrm{a} \to v) \to \mathrm{c} \to v,$

$G_v \coloneqq (\mathrm{a} \to \mathrm{t}) \to \mathrm{d} \to v,$

$G^v \coloneqq (\mathrm{a} \to v) \to \mathrm{e} \to \mathrm{t}.$



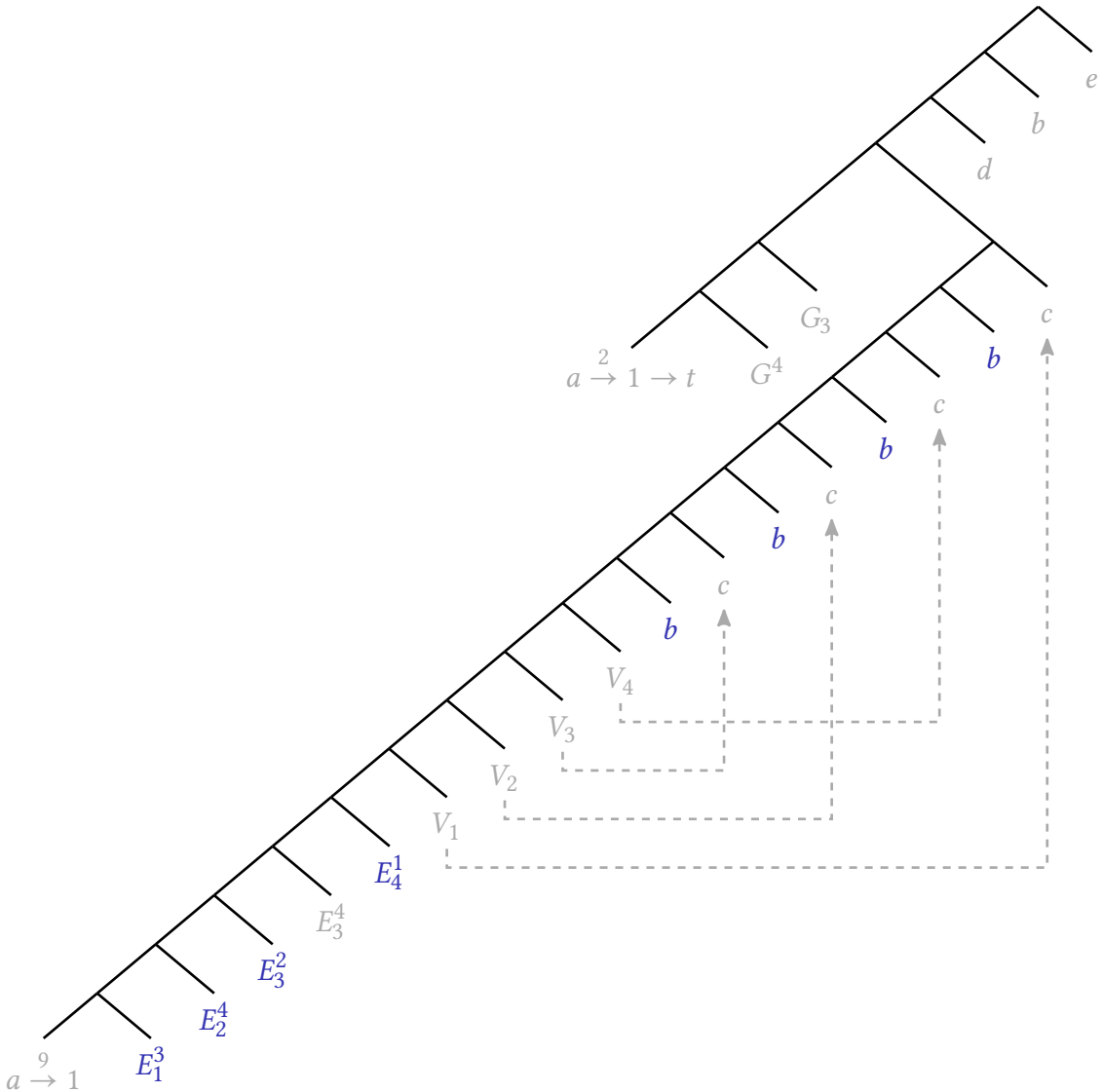
LEGEND

$E_u^v \coloneqq (\mathrm{a} \to u) \to \mathrm{b} \to v,$

$V_v \coloneqq (\mathrm{a} \to v) \to \mathrm{c} \to v,$

$G_v \coloneqq (\mathrm{a} \to \mathrm{t}) \to \mathrm{d} \to v,$

$G^v \coloneqq (\mathrm{a} \to v) \to \mathrm{e} \to \mathrm{t}.$



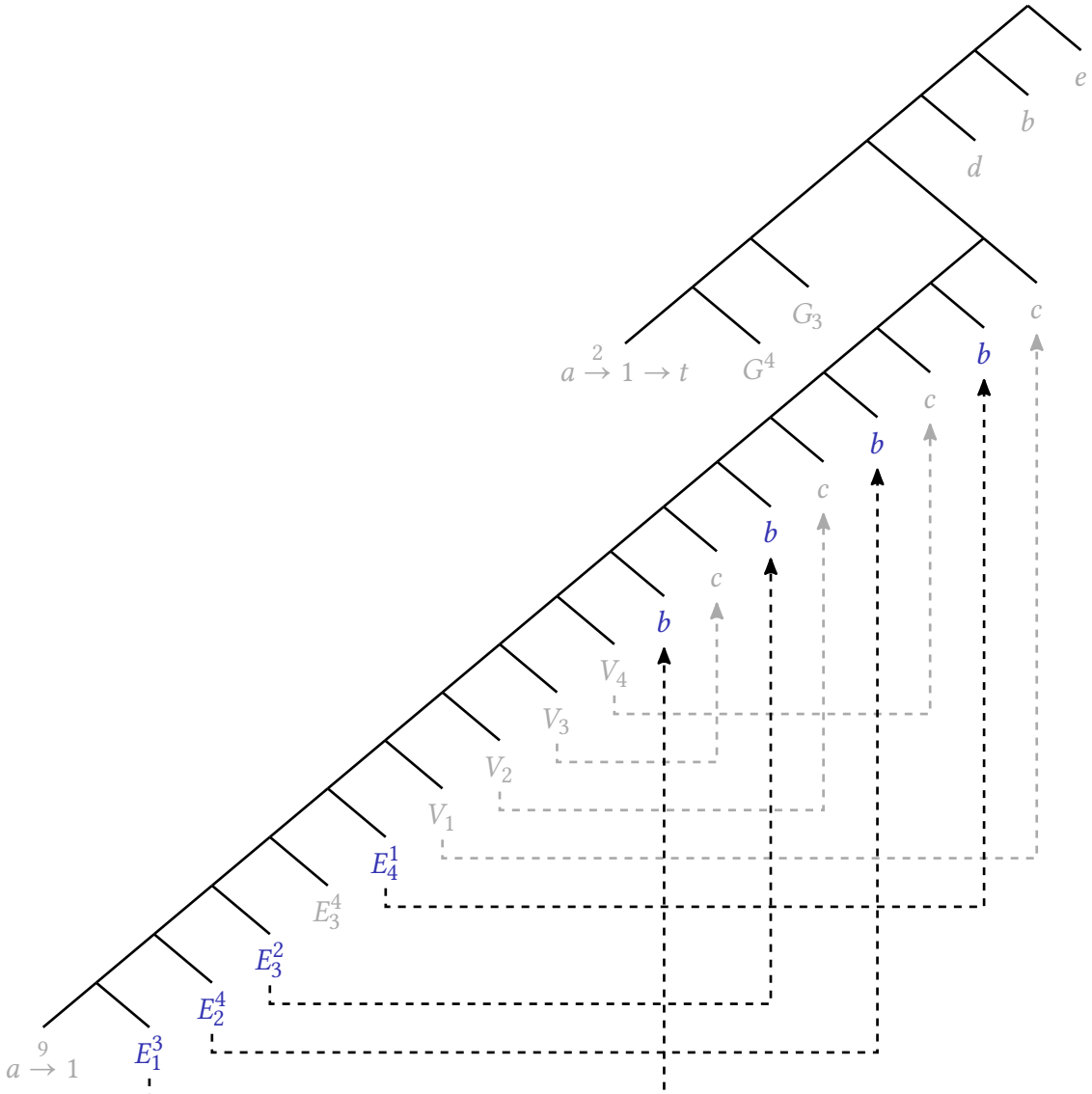
LEGEND

$E_u^v := (\mathrm{a} \to u) \to \mathrm{b} \to v,$

$V_v := (\mathrm{a} \to v) \to \mathrm{c} \to v,$

$G_v := (\mathrm{a} \to \mathrm{t}) \to \mathrm{d} \to v,$

$G^v := (\mathrm{a} \to v) \to \mathrm{e} \to \mathrm{t}.$



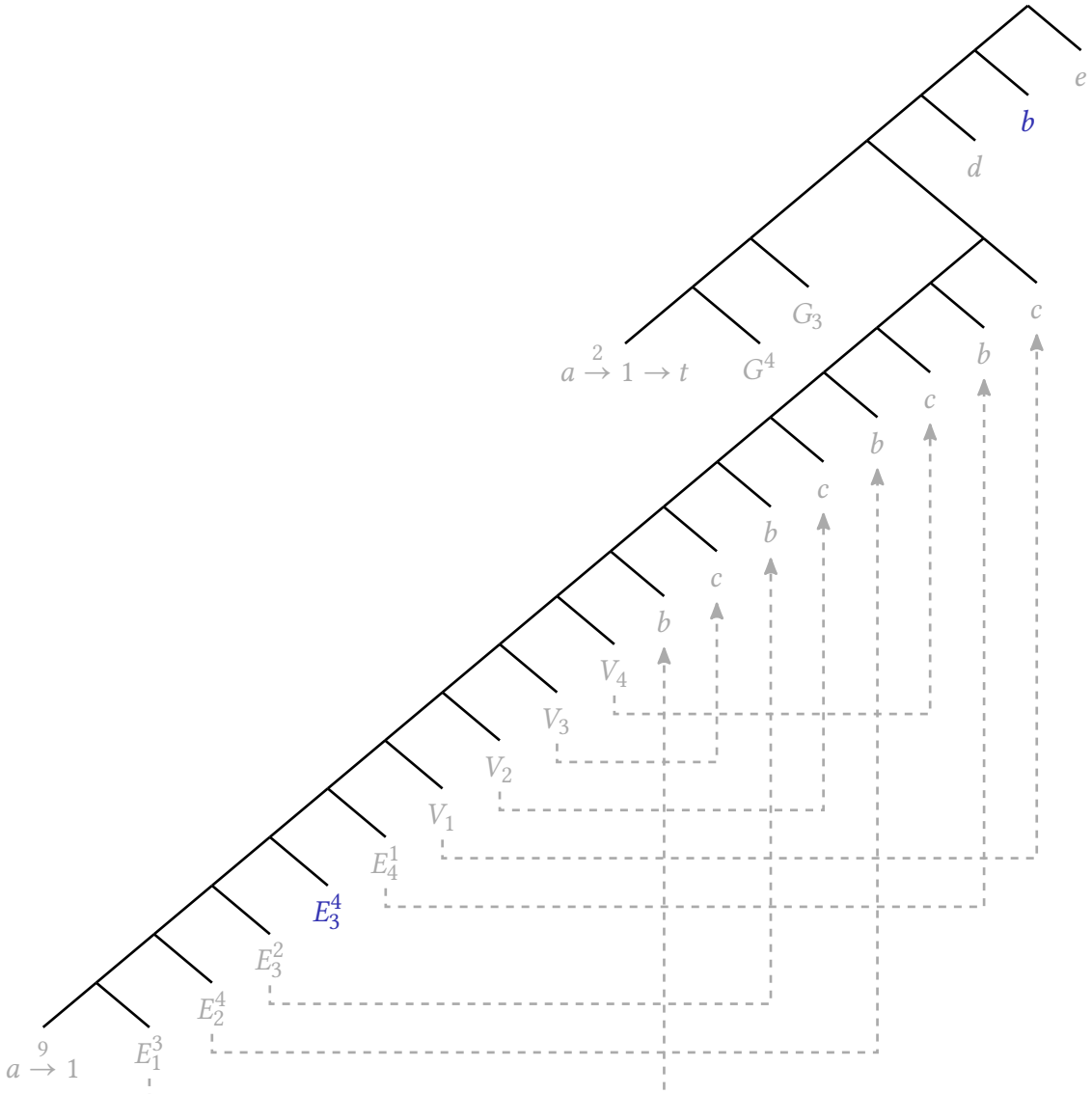
LEGEND

$E_u^v \coloneqq (\mathrm{a} \to u) \to \mathrm{b} \to v,$

$V_v \coloneqq (\mathrm{a} \to v) \to \mathrm{c} \to v,$

$G_v \coloneqq (\mathrm{a} \to \mathrm{t}) \to \mathrm{d} \to v,$

$G^v \coloneqq (\mathrm{a} \to v) \to \mathrm{e} \to \mathrm{t}.$



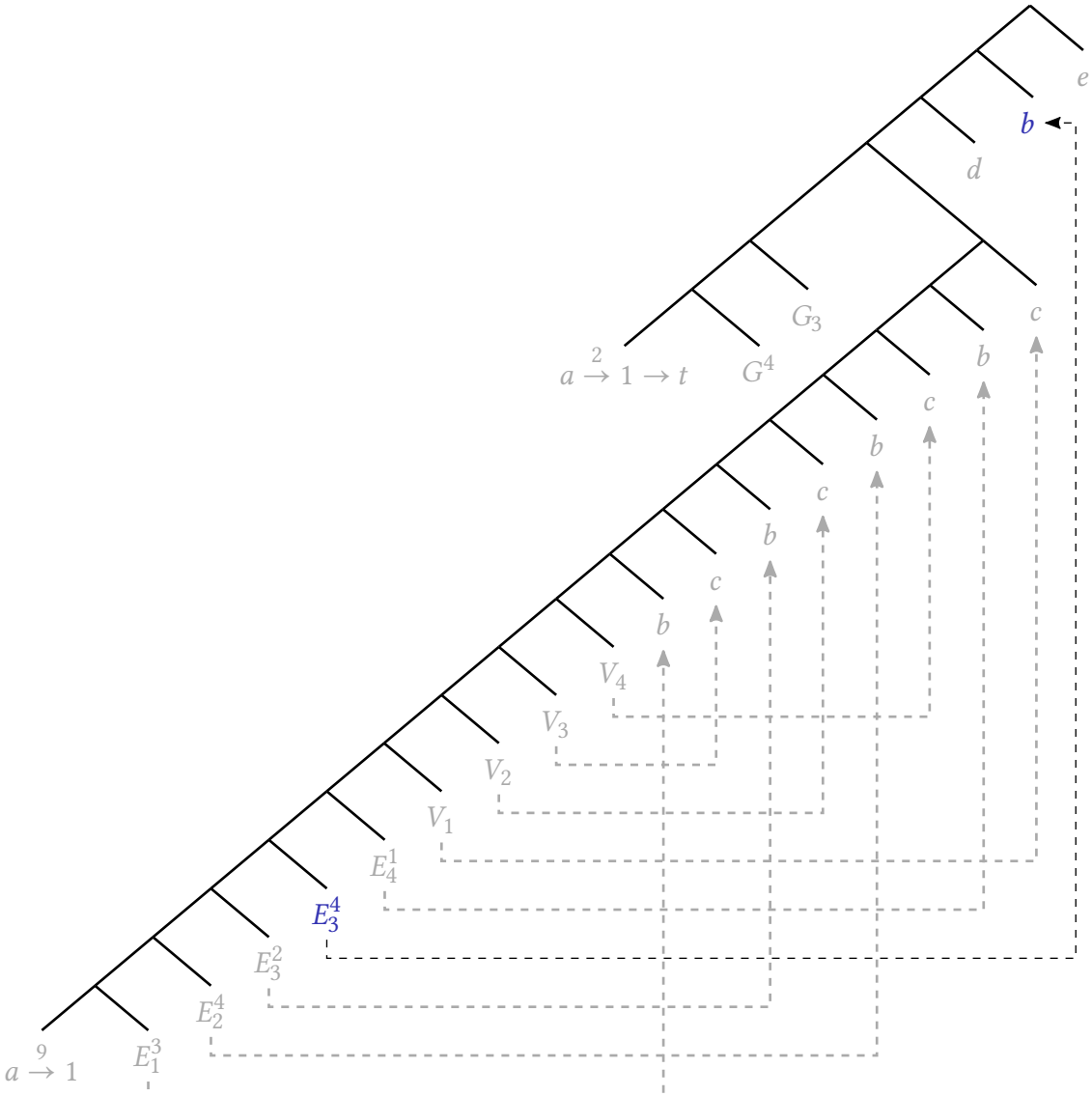
LEGEND

$E_u^v \coloneqq (\mathrm{a} \to u) \to \mathrm{b} \to v,$

$V_v \coloneqq (\mathrm{a} \to v) \to \mathrm{c} \to v,$

$G_v \coloneqq (\mathrm{a} \to \mathrm{t}) \to \mathrm{d} \to v,$

$G^v \coloneqq (\mathrm{a} \to v) \to \mathrm{e} \to \mathrm{t}.$



LEGEND

$E_u^v := (\mathrm{a} \to u) \to \mathrm{b} \to v,$

$V_v := (\mathrm{a} \to v) \to \mathrm{c} \to v,$

$G_v := (\mathrm{a} \to \mathrm{t}) \to \mathrm{d} \to v,$

$G^v := (\mathrm{a} \to v) \to \mathrm{e} \to \mathrm{t}.$

DISCUSSION

Discussion

Q: *Given a surface structure, can an LF be computed efficiently?*

Discussion

Q: *Given a surface structure, can an LF be computed efficiently?*

A: *No, because the problem is NP-complete*

Discussion

Q: *Given a surface structure, can an LF be computed efficiently?*
A: *No, because the problem is NP-complete*

There may exist good heuristics for finding LFs

Discussion

Q: *Given a surface structure, can an LF be computed efficiently?*
A: *No, because the problem is NP-complete*

There may exist good heuristics for finding LFs

But NP completeness guarantees* that they will fail on certain inputs

*Unless P = NP.

Discussion

Q: *Given a surface structure, can an LF be computed efficiently?*
A: *No, because the problem is NP-complete*

There may exist good heuristics for finding LFs

But NP completeness guarantees* that they will fail on certain inputs

*Unless P = NP.

No efficient program can automate away the task of finding LFs.

Other Scope-Taking Operations

Conjecture: Similar NP-completeness results for Quantifying In, Cooper storage, continuations/tower grammars, etc.

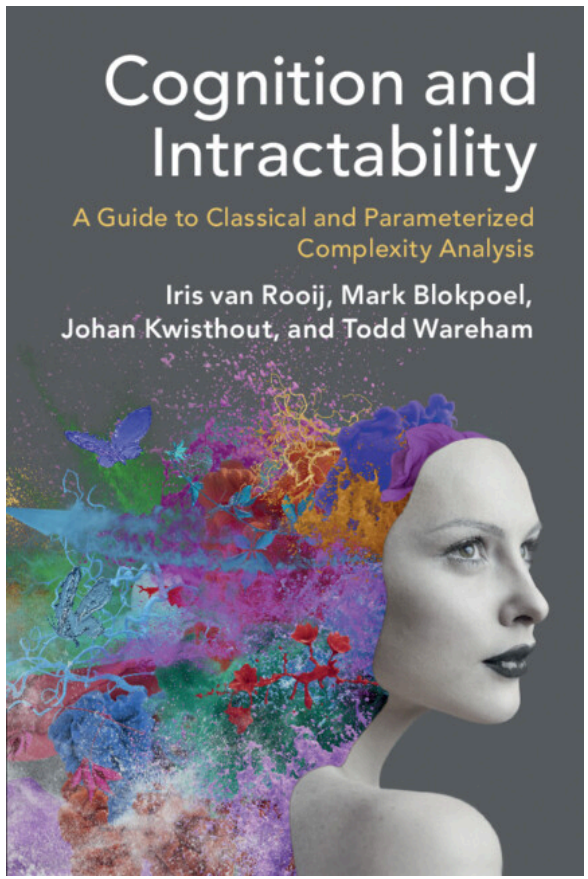
Other Scope-Taking Operations

Conjecture: Similar NP-completeness results for Quantifying In, Cooper storage, continuations/tower grammars, etc.

And the proof can use the same reduction from Directed Hamiltonian Cycle

Discussion

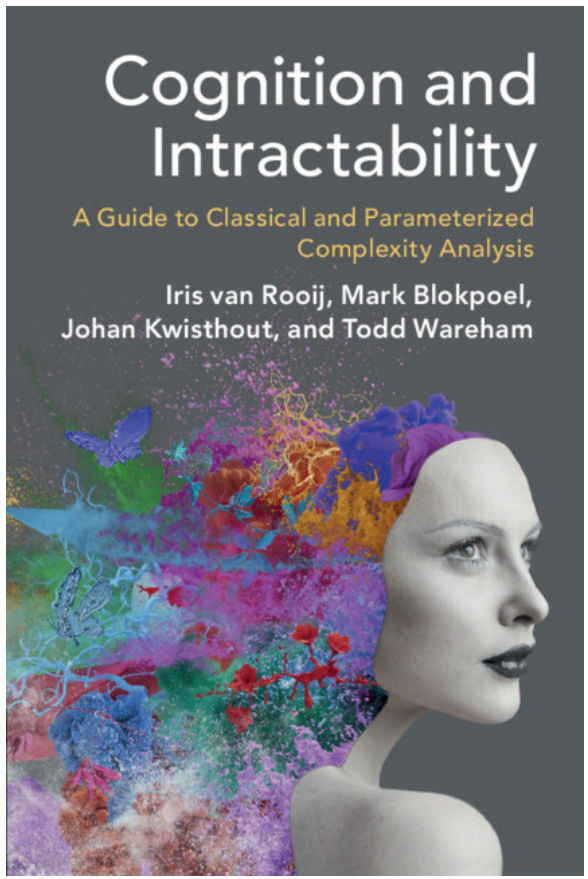
Some say NP-completeness is the right level of complexity for language (cf. Ristad 1993, Icard & Moss 2022)



Discussion

Some say NP-completeness is the right level of complexity for language (cf. Ristad 1993, Icard & Moss 2022)

Others in philosophy & cog sci think cognition is limited to *fixed-parameter tractable* problems

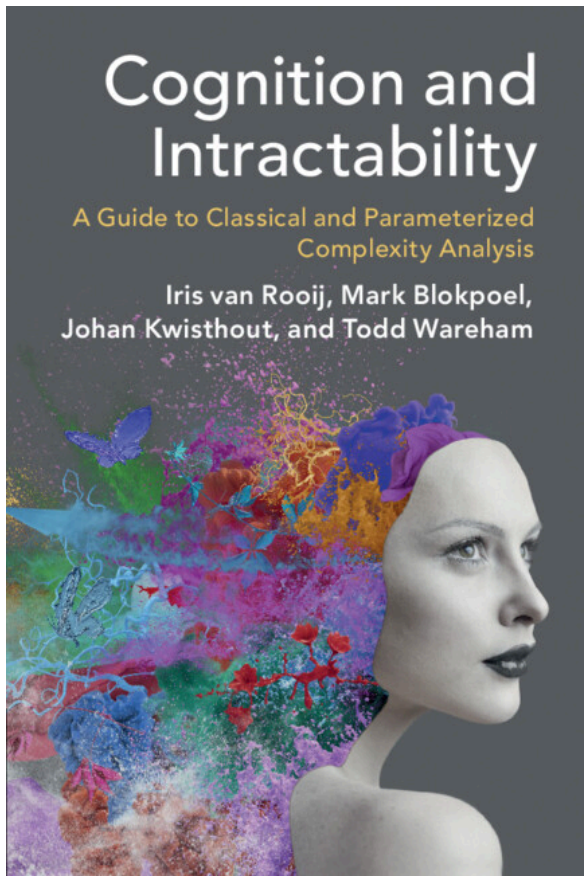


Discussion

Some say NP-completeness is the right level of complexity for language (cf. Ristad 1993, Icard & Moss 2022)

Others in philosophy & cog sci think cognition is limited to *fixed-parameter tractable* problems

Does LF Repair become tractable when certain parameters are held fixed?



Chasing Tractability

Chasing Tractability

Possible parameters:

- number of atomic types?
- order of traces/variables? (cf. Landman 2006, Poole 2024)

Chasing Tractability

Possible parameters:

- number of atomic types?
- order of traces/variables? (cf. Landman 2006, Poole 2024)

Possible constraints:

- no parasitic scope?
- no cyclic QR?
- only QR to the top of the tree?
  - should work if all types look like: $(\alpha \to \beta) \to \gamma$ with β, γ atomic

Shouldn't QR be easy?

Why, despite intractability, are LFs pretty easy to find in practice?

Shouldn't QR be easy?

Why, despite intractability, are LFs pretty easy to find in practice?

Seems relevant that the general type of a scope-taker is $(\alpha \to \beta) \to \gamma$, but in practice we often see $(\alpha \to \beta) \to \beta$

Shouldn't QR be easy?

Why, despite intractability, are LFs pretty easy to find in practice?

Seems relevant that the general type of a scope-taker is $(\alpha \to \beta) \to \gamma$, but in practice we often see $(\alpha \to \beta) \to \beta$

When $\beta \neq \gamma$, we can use types to track state and scope-taking to change state. This was the whole idea of the NP-completeness proof

Shouldn't QR be easy?

Why, despite intractability, are LFs pretty easy to find in practice?

Seems relevant that the general type of a scope-taker is $(\alpha \to \beta) \to \gamma$, but in practice we often see $(\alpha \to \beta) \to \beta$

When $\beta \neq \gamma$, we can use types to track state and scope-taking to change state. This was the whole idea of the NP-completeness proof

Some semantic formalisms use types to track state, e.g. any kind of variable-free semantics:

- proposition with a pronoun: type $e \Rightarrow t$
- quantifier that binds a pronoun in its scope: type $(e \to (e \Rightarrow t)) \to t$

Shouldn't QR be easy?

Why, despite intractability, are LFs pretty easy to find in practice?

Seems relevant that the general type of a scope-taker is $(\alpha \to \beta) \to \gamma$, but in practice we often see $(\alpha \to \beta) \to \beta$

When $\beta \neq \gamma$, we can use types to track state and scope-taking to change state. This was the whole idea of the NP-completeness proof

Some semantic formalisms use types to track state, e.g. any kind of variable-free semantics:

- proposition with a pronoun: type $e \Rightarrow t$
- quantifier that binds a pronoun in its scope: type $(e \to (e \Rightarrow t)) \to t$

This seems necessary for NP-completeness. (Maybe sufficient?)

Thanks!

Thanks!

(And thanks to CB, without whose work none of these questions could have been asked or answered.)

References

Barker, Chris, and Chung-chieh Shan. 2014. *Continuations and Natural Language*. Oxford University Press.

Barker, Chris. 2020. "The Logic of Quantifier Raising." *Semantics and Pragmatics* 13: 1–40.

Cooper, Robin. 1983. *Quantification and Syntactic Theory*. D. Reidel.

Hallman, Peter. 2023. "Superlative displacement in 'sandwich' scenarios." *Natural Language Semantics* 31: 1–23.

Heim, Irene, and Angelika Kratzer. 1998. *Semantics in Generative Grammar*. Blackwell Textbooks in Linguistics 13. Wiley Blackwell.

Hendriks, Herman. 1993. "Studied Flexibility." PhD thesis, University of Amsterdam.

Icard, Thomas F., and Larry S. Moss. 2023. “A Simple Logic of Concepts.” *Journal of Philosphical Logic* 52: 705–730.

Kiselyov, Oleg, and Chung-chieh Shan. 2014. “Continuation Hierarchy and Quantifier Scope.” In *Formal Approaches to Semantics and Pragmatics: Japanese and Beyond*, edited by Elin McCready, Katsuhiko Yabushita, and Kei Yoshimoto. Springer.

Landman, Meredith Lizabeth. 2006. “Variables in Natural Language.” PhD thesis, University of Massachusetts Amherst.

May, Robert. 1977. “The Grammar of Quantification.” PhD thesis, Massachusetts Institute of Technology.

Montague, Richard. 1973. “The Proper Treatment of Quantification in Ordinary English.” In *Approaches to Natural Language: Proceedings of the 1970 Stanford Workshop on*

Grammar and Semantics, edited by K. J. J. Hintikka, J. M. E. Moravcsik, and P. Suppes. Springer.

Poole, Ethan. 2024. “(Im)possible Traces.” *Linguistic Inquiry* 55 (2): 287–326.

Ristad, Eric Sven. 1993. *The Language Complexity Game*. MIT Press.

Rooij, Iris van, Mark Blokpoel, Johan Kwisthout, and Todd Wareham. 2019. *Cognition and Intractability: A Guide to Classical and Parameterized Complexity Analysis*. Cambridge University Press.